

PRESENTE

Plataforma de Comunicação Escola-Família

DOCUMENTO DE REQUISITOS DE PRODUTO

Versão	1.0 — MVP
Data	Abril 2026
Status	Em definição
Produto	Presente
Segmento	EdTech B2B — Escolas Publicas e Privadas

1. Visão Geral do Produto

1.1 Problema que Resolve

A comunicação entre escola e família é hoje feita majoritariamente por grupos de WhatsApp. Esse modelo gera cinco problemas críticos para as escolas:

- Informações importantes se perdem no ruído de dezenas de mensagens diárias
- A escola não tem como provar que o comunicado foi recebido e lido
- Pais que não estão no grupo — ou saem — ficam sem informação
- O celular do professor e da direção fica exposto a todos os responsáveis
- Não existe histórico organizado das comunicações por aluno ou turma

O Presente substitui esse modelo com uma plataforma profissional de comunicação e agenda escolar, com confirmação de leitura registrada e acesso organizado para pais, professores e gestão.

1.2 Proposta de Valor

Para a escola	Canal oficial de comunicação com registro de leitura, sem expor dados pessoais de ninguém
Para o professor	Comunica para a turma em segundos, sabe quem leu e quem não leu, sem WhatsApp pessoal
Para os responsáveis	Um lugar único e organizado com tudo que a escola mandou para o filho deles — sem ruído, sem informação perdida no grupo
Complemento digital à agenda física	A agenda física continua sendo útil para os alunos no dia a dia da sala de aula. O Presente não a substitui — é um canal complementar e digital de comunicação entre escola e pais. Os comunicados, avisos e eventos chegam em tempo real aos responsáveis com confirmação de leitura, eliminando o retrabalho de circulares impressas e grupos de WhatsApp para esse fim específico
Diferencial	Log de confirmação com data/hora/usuário — prova jurídica e administrativa de que a informação chegou

1.3 Posicionamento

Presente não é um sistema de gestão escolar (ERP) nem substitui o WhatsApp para comunicação livre e informal. É o canal oficial da escola — assimétrico e estruturado: a escola comunica via comunicados (avisos, fotos, documentos); os responsáveis respondem com comunicados de tipo específico (justificativas, atestados, saída antecipada). Toda interação é

classificada, confirmada e rastreável. A escola contrata, paga e configura. Os pais usam gratuitamente.

2. Personas

Persona 1 — Diretora / Coordenadora Pedagógica

Nome	Sônia, 44 anos
Cargo	Diretora de escola privada (120 alunos, Ed. Infantil e Fund. I)
Perfil	Pedagoga com 18 anos de experiência. Conhece bem o lado humano, mas evita tecnologia complexa
Dor principal	Pais que dizem 'não fui avisado' mesmo após comunicado no WhatsApp — sem nenhuma forma de provar
Objetivo	Ter uma ferramenta que profissionalize a comunicação sem precisar de treinamento extenso para a equipe
Comportamento	Toma decisão de compra rapidamente se vê que resolve uma dor real. Prefere demonstração prática a apresentação comercial
Frase típica	"No WhatsApp eu nunca sei quem realmente leu. Já tivemos problema com pai na reunião que disse que não sabia de nada."

Persona 2 — Professor(a) de Turma

Nome	Juliana, 31 anos
Cargo	Professora do 3º ano do Ensino Fundamental
Perfil	Usa o celular pessoal para tudo relacionado à turma. Está no grupo de WhatsApp dos responsáveis e recebe mensagem a qualquer hora
Dor principal	Perda de privacidade e invasão fora do horário de trabalho. Pais que mandam mensagem às 23h e esperam resposta
Objetivo	Comunicar com a turma de forma profissional sem expor seu número pessoal
Comportamento	Usuária frequente do produto — quem mais envia comunicados e usa a agenda. Adoção depende da facilidade de uso
Frase típica	"Meu número ficou exposto para 28 famílias. Às vezes recebo mensagem de madrugada sobre lição de casa."

Persona 3 — Pai / Mãe / Responsável

Nome	Ricardo, 38 anos
-------------	------------------

Perfil	Pai de dois filhos em escolas diferentes. Trabalha em tempo integral, acessa o celular pouco durante o dia
Dor principal	Perde comunicados importantes no meio das mensagens do grupo. Não sabe o que é urgente e o que não é
Objetivo	Ver rapidamente tudo que a escola mandou para o seu filho, organizado e sem ruído
Comportamento	Usuário passivo — acessa quando recebe notificação. Não quer aprender sistema complexo. Se o app não for simples, não usa
Frase típica	"Fico sabendo das coisas da escola pelos outros responsáveis no grupo, não pela escola. Às vezes fico por fora."

Persona 4 — Secretaria Escolar

Nome	Fernanda, 29 anos
Cargo	Secretária administrativa da escola
Perfil	Responsável por comunicados gerais, matrículas e documentos. Lida com pais difíceis no balcão todo dia
Dor principal	Pais que chegam na escola reclamando que não foram avisados de coisas que foram comunicadas
Objetivo	Ter histórico documentado de toda comunicação para usar como evidência em conflitos com famílias
Comportamento	Usuária administrativa — cria comunicados gerais, faz relatórios de leitura, gerencia a base de famílias
Frase típica	"Se o responsável diz que não recebeu, hoje não tenho como provar que mandei. Com isso vira discussão."

3. Requisitos Funcionais

3.1 Módulo de Gestão de Usuários e Turmas

RF-01 — Cadastro da Escola (Administrador SaaS)

- O cadastro da escola é realizado exclusivamente pelo administrador do SaaS — não pela própria escola. Esse fluxo cria o tenant, define o plano e gera as credenciais do primeiro acesso
- Admin SaaS cadastra: nome da escola, CNPJ, plano contratado, dados de cobrança recorrente (cartão, boleto ou PIX) e gera o e-mail + senha temporária do administrador da escola
- Admin SaaS define o número máximo de unidades permitidas pelo plano. A escola recebe por e-mail as credenciais do primeiro acesso e uma URL personalizada do painel

RF-01b — Gestão da Escola (Administrador da Escola)

- Após o primeiro acesso com as credenciais fornecidas pelo SaaS, o administrador da escola completa o perfil: logo, endereço, telefone e dados de contato
- Gestão de unidades: admin cadastra as unidades da escola (nome, endereço, telefone) dentro do limite do plano
- Gestão hierárquica de usuários: admin cadastra Diretor → Diretor cadastra Coordenadores → Coordenador cadastra Professores — cada nível só enxerga e gerencia o nível abaixo
- Gestão de turmas: admin ou coordenador cria turmas vinculadas a uma unidade, com nome, série, turno e ano letivo

RF-02 — Gestão de turmas

- Administrador cria turmas com nome, nível de ensino, turno e ano letivo
- Turmas têm um ou mais professores responsáveis associados
- Turmas podem ser arquivadas ao final do ano letivo sem perda de histórico

RF-03 — Cadastro de alunos

- Aluno é cadastrado com nome, turma, data de nascimento e observações
- Cada aluno pode ter até 5 responsáveis vinculados (pai, mãe, avó, etc.)
- Importação em lote via planilha CSV/Excel — módulo inteligente de importação (ver seção 3.2). Um aluno pode estar vinculado a mais de uma turma simultaneamente (ex: turma regular + turma de reforço)
- Cada responsável recebe convite individual por SMS ou link

RF-04 — Perfis e permissões

Perfil	Permissões	Acesso
Administrador	Tudo — configuração, usuários, relatórios, financeiro	Web + app
Professor	Comunicados da sua turma, agenda da sua turma, lista de alunos	Web + app
Secretaria	Comunicados gerais, base de famílias, relatórios de leitura	Web + app
Responsável (responsável)	Leitura de comunicados e agenda do seu filho, confirmação de leitura, e envio de comunicados do tipo responsável (atestado, justificativa de falta, saída antecipada) referentes ao seu filho	App mobile + web (visibilidade restrita ao seu filho)

3.2 Módulo de Importação e Onboarding Colaborativo

A escola não precisa ter os dados completos para começar. O Presente foi desenhado para aceitar dados parciais e deixar que os próprios responsáveis completem e validem as informações — num modelo colaborativo onde a escola estrutura e os responsáveis confirmam.

RF-05 — Importação Inteligente por Planilha

- Aceita exportações dos principais SGEs brasileiros (Escola Web, SAE Digital, RM Totvs, Lyceum) e qualquer planilha CSV/Excel. Importação disponível para três entidades: turmas, professores e alunos — cada uma com seu template específico
- Detecção automática de campos: o sistema reconhece "Responsável 1", "Nome do Pai", "resp1" ou "guardian_name" como o mesmo campo — sem mapeamento manual pela secretaria
- Importação tolerante a dados incompletos: planilha sem telefone, sem e-mail ou com campos em branco é aceita normalmente — o sistema sinaliza o que está faltando sem bloquear o processo
- Pré-visualização antes de confirmar: escola vê um resumo do que será importado (X alunos, Y turmas, Z responsáveis com telefone, W sem telefone) e confirma antes de salvar
- Reimportação segura: atualiza dados existentes sem duplicar registros — alunos cruzados por matrícula, professores por e-mail, turmas por nome+ano letivo

RF-06 — Onboarding Colaborativo dos Responsáveis

- Dois caminhos de ativação: (1) escola importa e dispara convite por SMS — responsável clica no link, valida OTP e ativa; (2) responsável baixa o app, busca a escola por nome ou CNPJ, seleciona a turma do filho, preenche seus dados e aguarda aprovação da escola no painel administrativo
- Na ativação, o responsável vê os dados que a escola importou sobre ele e pode corrigir: nome, e-mail, telefone e grau de parentesco — o Presente passa a ser a fonte da verdade para esses dados de contato

- Responsável pode convidar outros responsáveis do mesmo aluno diretamente pelo app (mãe, pai, avó, padrasto) — cada um valida o próprio telefone e se vincula ao aluno sem passar pela secretaria
- Responsáveis sem telefone na importação ficam na fila "pendentes" — a escola recebe uma lista para acionar manualmente (bilhete na mochila, abordagem na entrada) e pode inserir o telefone manualmente para reativar o convite

RF-07 — Vinculação e Validação de Responsáveis

- Vínculo automático: se o telefone ou nome do responsável bate com dado importado, vínculo aprovado sem intervenção. Auto-cadastro via busca no app sempre requer aprovação manual da escola — responsável fica com status PENDING_APPROVAL até a secretaria ou admin confirmar
- Vínculo manual: quando há divergência entre os dados (nome diferente, telefone não estava na planilha), o sistema segura a ativação e notifica a secretaria para aprovar ou rejeitar — proteção contra vínculos indevidos
- Segundo responsável convidado pelo responsável: vínculo aprovado automaticamente se o primeiro responsável já está validado — a escola é notificada mas não precisa aprovar, podendo revogar se necessário
- Dashboard de adesão: escola acompanha em tempo real quantos responsáveis já ativaram o acesso por turma — permite ação proativa antes do lançamento oficial para os responsáveis

3.3 Módulo de Comunicados

RF-08 — Criação de comunicado

- Tipos de conteúdo suportados no MVP: texto formatado e até 2 imagens por comunicado (JPG/PNG). PDF, áudio e vídeo são roadmap pós-MVP
- Comunicado multimídia em tempo real: professor pode registrar e compartilhar momentos da aula (atividade, passeio, projeto) com fotos tiradas na hora — pais acompanham o que acontece sem precisar esperar o boletim ou a reunião
- Destinatários: turma específica, múltiplas turmas, toda a escola, aluno individual, responsável individual
- Opção de agendar envio para data e hora futura
- Classificação por categoria: Aviso, Lembrete, Urgente, Informativo, Documento
- Comunicados marcados como Urgente geram notificação push prioritária e não ficam silenciados

RF-09 — Confirmação de leitura

- Comunicados podem ter dois modos de confirmação: (1) Ciência — botão "Li e estou ciente"; (2) Autorização — checkbox obrigatório "Li e estou ciente" + botões "Autorizo" ou "Não autorizo". O modo é definido pelo criador do comunicado no momento do envio

- Sistema registra: usuário, data, hora, dispositivo, versão do app e geolocalização (quando permitida pelo responsável) de cada confirmação — dados usados para auditoria e validade jurídica
- Escola vê em tempo real: enviado para X, lido por Y, pendente com Z
- Reenvio automático configurável: lembrar não-lidos após 24h, 48h ou data limite
- Exportação do relatório de leitura em PDF

RF-10 — Feed do responsável

- Responsável vê feed cronológico de todos os comunicados do(s) seu(s) filho(s)
- Filtros por filho (se tiver mais de um na escola), data e categoria
- Comunicados não lidos aparecem destacados no topo
- Documentos recebidos ficam organizados em aba separada e acessíveis offline

RF-11 — Comunicação Assimétrica e Estruturada

- A comunicação é assimétrica por design: escola envia comunicados via formulários predefinidos por categoria (Aviso, Urgente, Informativo, Documento, Foto, Prova, Reunião), podendo adicionar texto livre e até 2 fotos em cada um; responsáveis comunicam somente via tipos predefinidos (atestado, justificativa, saída antecipada). Visibilidade por perfil: Secretaria e Coordenadora enviam para todas as turmas em lote; Professor envia apenas para sua(s) turma(s); Responsável envia somente referente ao seu filho, visível para professor titular e coordenadora da turma
- Responsável pode abrir um formulário a qualquer momento pelo app (menu fixo) ou em resposta a um comunicado específico da escola — dependendo do tipo de ocorrência
- Toda comunicação do responsável para a escola é protocolada automaticamente com número, data, hora e status (recebido, em análise, resolvido) — gerando histórico auditável
- A escola define quais formulários ficam disponíveis para os responsáveis — podendo ativar, desativar e customizar os formulários padrão ou criar os seus próprios

3.4 Módulo de Comunicados do Responsável

RF-12 — Comunicados do Tipo Responsável (justificativas, atestados)

- Justificativa de falta: responsável informa o(s) dia(s) ausente(s), motivo (doença, viagem, emergência, outro) e campo opcional para observação de até 300 caracteres
- Envio de atestado médico: upload de foto ou PDF do atestado, com data de emissão, CRM do médico e período de afastamento — enviado direto ao prontuário do aluno na plataforma
- Autorização para saída antecipada: data, horário previsto de saída, nome e CPF do responsável que buscará a criança e motivo — gera registro impresso disponível na secretaria
- Responsável com filhos em mais de uma escola: ao enviar uma justificativa ou atestado, pode selecionar múltiplos filhos de escolas diferentes no mesmo envio — o sistema

distribui o comunicado para cada escola correspondente com um protocolo único por escola

RF-13 — Gestão e Resolução de Comunicados do Responsável

- Administrador cria formulários personalizados com campos de: seleção única, múltipla escolha, data, upload de arquivo e texto curto (até 300 caracteres)
- Cada formulário é ativado/desativado por turma ou para toda a escola — escola controla quais formulários ficam visíveis para os responsáveis a cada momento
- Respostas dos formulários chegam ao responsável na secretaria ou do professor responsável com notificação — nunca no celular pessoal de ninguém

RF-14 — Protocolo e Gestão de Solicitações

- Cada formulário enviado gera automaticamente um número de protocolo único — responsável acompanha o status pelo app: Recebido, Em análise, Resolvido
- Escola pode registrar uma nota interna ao resolver a solicitação — visível apenas para a equipe, não para o responsável
- Histórico de todas as solicitações por aluno — acessível pela secretaria a qualquer momento, exportável em PDF para fins administrativos ou jurídicos

3.5 Módulo de Agenda Escolar

RF-15 — Criação de eventos

- Eventos com: título, data, horário, local, descrição, turmas envolvidas
- Tipos: Prova/Avaliação, Reunião de Pais, Passeio, Feriado Escolar, Evento Cultural, Outro
- Evento de prova vincula matéria e turma — aparece destacado na agenda do aluno
- Recorrência: evento que se repete semanalmente/mensalmente (ex: reunião mensal de responsáveis)

RF-16 — Visão do responsável na agenda

- Calendário mensal filtrado para o(s) filho(s) do responsável
- Visualização em lista (próximos 30 dias) ou calendário mensal
- Notificação push 24h e 2h antes de eventos marcados como importantes
- Responsável pode adicionar evento ao calendário pessoal (Google Calendar / Apple Calendar)

3.6 Módulo Administrativo do SaaS (Super Admin)

Visível exclusivamente para o administrador do SaaS — não acessível pelas escolas. Centraliza a gestão de todos os tenants, monitoramento de uso e saúde financeira da plataforma.

RF-20 — Painel de Escolas Cadastradas

- Lista de todas as escolas com: nome, CNPJ, plano, data de início, status (trial, ativo, inadimplente, suspenso)
- Ações rápidas: suspender acesso, alterar plano, resetar senha do admin da escola, ver logs
- Filtros por plano, status, data de cadastro e região

RF-21 — Indicadores por Escola

Para cada escola, o SaaS admin visualiza os seguintes indicadores em tempo real:

Indicador	Descrição e impacto no custo
Total de alunos ativos	Base de cobrança — determina plano e valor mensal
Total de responsáveis ativos	Push tokens registrados — indica adoção real do app
Comunicados enviados/mês	Volume de uso — identifica escolas que usam muito vs pouco
SMS de OTP enviados/mês	Custo direto de SMS — alta rotatividade gera mais OTPs
Storage S3 utilizado (GB)	Custo de armazenamento — escolas com muitas fotos impactam mais
Push notifications/mês	Volume FCM — gratuito mas indica engajamento
Último acesso ao painel	Risco de churn — escola inativa há +30 dias merece atenção
Taxa de leitura média	Indicador de saúde do produto naquela escola

RF-22 — Compatibilidade de Preço por Escola

- Custo de infraestrutura calculado automaticamente por escola (SMS + S3 + parcela proporcional Fargate/RDS) versus receita do plano
- Alerta quando margem de uma escola específica cai abaixo de 70% — uso intenso ou plano subdimensionado
- Sugestão automática de upgrade quando escola ultrapassa 80% do limite de alunos do plano
- Histórico de evolução de uso mês a mês para planejar upgrades proativamente

RF-23 — Gestão Financeira SaaS

- Status de cobrança por escola: em dia, vencido, inadimplente — com data e valor
- Histórico de pagamentos e faturas por escola
- Suspensão automática configurável após X dias de inadimplência
- Dashboard de MRR total, por plano e evolução mensal
- Churn rate mensal — escolas que cancelaram ou foram suspensas

RF-24 — Logs e Auditoria Global

- Log de todas as ações do SaaS admin: cadastros, alterações de plano, suspensões
- Acesso a logs de erro por escola para suporte técnico
- Relatório de uptime e incidentes por período

3.7 Módulo Administrativo (Escola)

RF-17 — Dashboard da escola

- Indicadores: total de comunicados enviados, taxa de leitura geral, comunicados pendentes, eventos próximos
- Alerta de responsáveis que nunca acessaram o app (para ação proativa da secretaria)
- Ranking de engajamento por turma — quais turmas têm mais pais ativos

RF-18 — Relatórios

- Relatório de leitura por comunicado: quem leu, quando, em qual dispositivo
- Relatório de engajamento por responsável: quantos comunicados leu, quantos está devendo
- Exportação em PDF e Excel para fins administrativos e jurídicos

RF-19 — Gestão de acesso de responsáveis

- Secretaria vê lista de responsáveis com status: ativo, convidado (não aceitou ainda), inativo
- Reenvio de convite com 1 clique para quem ainda não ativou
- Remoção de acesso ao desligar aluno (transferência, formatura)

4. Requisitos Não Funcionais

4.1 Performance

- Notificação push entregue em até 5 segundos após envio do comunicado
- Carregamento do feed do responsável em menos de 2 segundos em 3G
- Suporte a 10.000 usuários simultâneos sem degradação (escala horizontal)
- Documentos PDF de até 20MB — processados e disponibilizados em até 30 segundos

4.2 Segurança e Privacidade

- Conformidade total com a LGPD (Lei 13.709/2018)
- Dados armazenados em servidores no Brasil (AWS São Paulo ou equivalente)
- Comunicação criptografada end-to-end (TLS 1.3)
- Dados de menores de idade: nenhum dado de aluno exposto para terceiros
- Números de celular de professores e staff nunca são visíveis para responsáveis
- Autenticação por SMS OTP (sem senha para o responsável — reduz abandono)
- Autenticação por e-mail + senha + 2FA opcional para perfis da escola
- Logs de acesso e ação armazenados por 5 anos (requisito para fins jurídicos)

4.3 Disponibilidade e Confiabilidade

- SLA de 99,5% de uptime mensal (acordado em contrato)
- Modo offline para responsáveis: comunicados já baixados acessíveis sem internet
- Fila de envio assíncrono — comunicado enfileirado mesmo se houver instabilidade
- Backup automático diário com retenção de 90 dias

4.4 Usabilidade

- Responsável consegue se cadastrar e ler o primeiro comunicado em menos de 3 minutos
- App funciona em dispositivos com Android 8+ e iOS 14+
- Interface acessível: suporte a texto grande (WCAG AA)
- Sem necessidade de treinamento formal para professores — onboarding in-app guiado
- Suporte a português brasileiro; inglês e espanhol como expansão futura

4.5 Escalabilidade

- Arquitetura multi-tenant: cada escola é isolada em nível de dados

- Plano de escola pode ser expandido sem migração de dados (upgrade de plano)
- API pública documentada para integração com sistemas de gestão escolar existentes

5. Regras de Negócio

5.1 Modelo de Acesso e Cobrança

RN-01	A escola é o único cliente pagante. Responsáveis e professores nunca pagam.
RN-02	Cobrança por aluno ativo no mês, independente do número de responsáveis vinculados.
RN-03	Aluno transferido ou formado é desativado — não conta na cobrança do mês seguinte.
RN-04	Período de teste gratuito de 30 dias sem cartão de crédito para escolas novas.
RN-05	Escola com plano vencido entra em modo leitura por 7 dias antes de suspensão total.
RN-06	O Presente é a fonte da verdade para dados de comunicação (telefone, e-mail, preferências). Dados acadêmicos (matrícula, turma, notas) pertencem ao SGE da escola.
RN-07	Dados de contato corrigidos pelo responsável no app têm precedência sobre os dados da planilha importada — a correção do responsável é sempre considerada mais atual e confiável.

5.2 Comunicação

RN-06	Comunicado enviado não pode ser editado — apenas cancelado e reemitido (preserva integridade do log).
RN-07	Comunicado cancelado continua visível no histórico com status "Cancelado" e data do cancelamento.
RN-08	Responsável só vê comunicados de turmas nas quais seus filhos estão atualmente matriculados.
RN-09	Comunicado com categoria "Urgente" não pode ser silenciado pelo responsável.
RN-10	Lembretes de não-lidos são enviados exclusivamente via push FCM. Máximo 2 lembretes por comunicado por responsável.
RN-11	Responsáveis comunicam com a escola exclusivamente via formulários estruturados predefinidos — nunca por texto livre. Isso garante rastreabilidade e elimina ruído operacional.
RN-12	Formulário enviado não pode ser editado ou cancelado pelo responsável — apenas pela escola, com registro da alteração e motivo.

RN-13	Formulários do responsável chegam ao responsável da secretaria ou professor responsável pela turma — nunca como mensagem no celular pessoal de nenhum funcionário.
--------------	--

5.3 Dados e Privacidade

RN-12	Dados de alunos menores de 18 anos não podem ser exportados sem anonimização, exceto pela própria escola.
RN-13	Ao cancelar contrato, escola tem 30 dias para exportar todos os dados antes da exclusão definitiva.
RN-14	Responsável pode solicitar exclusão dos seus dados pessoais — mantendo apenas o registro de leitura anonimizado.
RN-15	Log de confirmação de leitura é imutável — não pode ser alterado nem pela escola nem pela plataforma.

5.4 Agenda

RN-16	Eventos só podem ser criados para datas futuras (a partir de hoje).
RN-17	Evento passado pode ser editado apenas pelo administrador, não pelo professor.
RN-18	Cancelamento de evento gera notificação automática para todos os responsáveis das turmas envolvidas.

6. Integrações

6.1 Integrações do MVP (obrigatórias)

Notificações Push

Provedor	Firestore Cloud Messaging (FCM) — Android e iOS
Uso	Canal exclusivo para todos os comunicados, lembretes de agenda e alertas. Gratuito e ilimitado via FCM.
Fallback	SMS via Twilio ou Zenvia quando o usuário não tem o app instalado ou com notificação desativada

SMS (Convite e OTP)

Provedor sugerido	Zenvia (Brasil) — melhor custo e entrega nacional
Uso	Exclusivamente para OTP de ativação do responsável. Não é usado para comunicados ou lembretes.
Volume estimado	1-2 SMS por responsável (apenas OTP de ativação inicial)

Armazenamento de Arquivos

Provedor	AWS S3 (região sa-east-1 — São Paulo)
Uso	PDFs, imagens e áudios enviados nos comunicados
Regra	Arquivos associados a um tenant ficam em bucket isolado por escola

6.2 Integrações Pós-MVP (planejadas)

Integração	Descrição	Prioridade
Google Calendar	Responsável adiciona evento escolar diretamente ao Google Calendar com 1 toque	Alta
Apple Calendar	Mesmo que Google Calendar para usuários iOS	Alta
WhatsApp Business API	Canal alternativo futuro para responsáveis sem smartphone — feature premium opcional	Alta

Sistemas de Gestão Escolar (SGE)	API para importar base de alunos e turmas de sistemas como Escola Web, RM Totvs, etc.	Média
Google Workspace / Microsoft 365	SSO para professores que já usam e-mail Google ou Microsoft da escola	Média
PIX / Asaas	Gestão de cobranças de mensalidades (módulo futuro)	Baixa

6.3 API Pública

O Presente disponibilizará uma API REST documentada (OpenAPI 3.0) para:

- Importação automatizada de alunos e turmas a partir de sistemas legados
- Recebimento de webhooks de confirmação de leitura (para escolas com sistema próprio)
- Consulta de relatórios de engajamento por integração com BI da escola

7. Fluxos Principais de Uso

7.1 Fluxo de Onboarding da Escola

Passo 1	Administrador acessa o site, inicia trial gratuito de 30 dias
Passo 2	Preenche dados da escola e cria sua conta de administrador
Passo 3	Cria as turmas do ano letivo
Passo 4	Exporta dados do SGE ou prepara planilha com os dados que tiver (incompletos são aceitos). Faz upload no Presente — detecção automática de campos, pré-visualização e confirmação
Passo 5	Presente dispara SMS de convite automaticamente para responsáveis com telefone. Escola recebe lista de "pendentes" (sem telefone) para acionar por outros meios
Passo 6	Pais ativam o acesso, corrigem os próprios dados e convidam outros responsáveis. Escola acompanha taxa de adesão por turma no dashboard
Passo 7	Cria conta dos professores, associa às turmas e envia primeiro comunicado — escola e pais já estão conectados
Tempo estimado	Escola configurada e funcional em menos de 40 minutos. Adesão completa dos responsáveis em 3 a 7 dias úteis após os convites

7.2 Fluxo de Envio de Comunicado

Ator	Professor ou Secretaria
Passo 1	Acessa o responsável e clica em "Novo Comunicado"
Passo 2	Escreve o texto, anexa arquivo se necessário, seleciona categoria
Passo 3	Escolhe destinatários: turma, múltiplas turmas, escola inteira ou responsável específico
Passo 4	Configura se aceita resposta e se quer lembrete automático para não-lidos
Passo 5	Publica — notificação push enviada em até 5 segundos para todos os destinatários
Passo 6	Escola acompanha em tempo real quem confirmou leitura no responsável

7.3 Fluxo do Responsável (Responsável)

Ator	Responsável cadastrado pela escola
-------------	------------------------------------

Ativação	Recebe SMS com link de convite, instala app, valida telefone com OTP — sem senha
Uso diário	Abre app, vê feed com comunicados do filho. Não-lidos aparecem no topo com destaque
Leitura	Abre comunicado, lê, toca em "Li e estou ciente" — confirmação registrada
Agenda	Aba de agenda mostra próximos eventos do filho em ordem cronológica
Notificações	Recebe push para novos comunicados, lembretes de eventos e reenvios de não-lidos

8. Modelo de Precificação Sugerido

Plano	Alunos	Preço/mês	Destaques
Starter	Até 100	R\$ 199	Todas as funcionalidades do MVP
Escola	101 a 400	R\$ 449	API básica + suporte prioritário
Rede	401 a 1000	R\$ 890	Múltiplas unidades + API completa + CS dedicado
Enterprise	Acima de 1000	Sob consulta	SLA customizado + integrações dedicadas

Nota: precificação por aluno ativo/mês garante que a escola paga proporcional ao uso real. Não há cobrança por número de professores, responsáveis ou comunicados enviados.

9. Arquitetura de Nuvem e Análise Financeira

9.1 Stack Tecnológico Recomendado

Escolhas baseadas em custo-benefício para o Brasil, ecossistema unificado e menor curva de aprendizado para um time enxuto.

Camada	Tecnologia	Justificativa
App Mobile	React Native	Nativo iOS+Android, notificações confiáveis, câmera, modo offline
Web (escola)	React + Next.js	Mesmo ecossistema do mobile, reuso de componentes
Backend	Node.js + TypeScript	Linguagem única em todo o stack, time menor e mais produtivo
Banco de dados	PostgreSQL (AWS RDS)	Multi-tenant robusto, backup automático, LGPD-compliant
Cache / Sessões	ElastiCache Redis	Sessões de usuário, rate limiting, fila de notificações
Arquivos	AWS S3 (sa-east-1)	Região São Paulo, custo baixo, CDN integrado
Push	Firebase FCM	Gratuito, confiável, Android + iOS
SMS	Zenvia	Exclusivo para OTP de ativação. Comunicados via push FCM — gratuito e ilimitado
E-mail	AWS SES	R\$ 0,50 por 1.000 e-mails, alta entregabilidade

9.2 Infraestrutura Fixa Mensal (Compartilhada entre todos os clientes)

R\$ 667/mês fixos divididos entre todos os clientes ativos — quanto mais clientes, menor o custo por escola.

Componente	Custo/mês (USD)	Custo/mês (R\$)
Fargate — 2 tasks (0,5 vCPU + 1GB RAM cada)	~ US\$ 36	R\$ 184
RDS PostgreSQL db.t4g.small Multi-AZ	~ US\$ 45	R\$ 230
ALB — Application Load Balancer	~ US\$ 18	R\$ 92
ElastiCache Redis t4g.micro	~ US\$ 12	R\$ 61
Route53 + CloudWatch + Secrets Manager	~ US\$ 10	R\$ 51
Firebase FCM (push notifications)	Gratuito	R\$ 0

Apple Developer + Google Play + Domínio	~ US\$ 10	R\$ 50
TOTAL FIXO MENSAL	~ US\$ 131	R\$ 667

Câmbio: US\$ 1 = R\$ 5,10 (conservador). Variações cambiais podem impactar os custos AWS em até ±15%.

9.3 Custos Variáveis por Plano (por escola/mês)

Componente	Starter (100 al.)	Escola (250 al.)	Rede (700 al.)
S3 — Storage + Requests	R\$ 0,06	R\$ 0,13	R\$ 0,34
CloudFront — CDN	R\$ 0,30	R\$ 0,30	R\$ 0,30
SMS Zenvia (apenas OTP de ativação — amortizado)	R\$ 1,20	R\$ 3,00	R\$ 8,40
AWS SES (e-mails transacionais)	R\$ 0,28	R\$ 0,69	R\$ 1,93
TOTAL VARIÁVEL / escola	R\$ 1,86	R\$ 4,09	R\$ 10,97

9.4 Análise de Margem por Cenário de Crescimento

O custo fixo de R\$ 667/mês dilui com a escala — principal alavanca de margem em negócios SaaS.

Cenário	MRR	Custo Infra	Lucro Bruto	Margem
MVP — 7 clientes (5 Starter + 2 Escola)	R\$ 1.893	R\$ 788	R\$ 1.105	58,3%
Conservador — 21 clientes (~6 meses)	R\$ 6.120	R\$ 1.085	R\$ 5.035	82,3%
Moderado — 59 clientes (~12 meses)	R\$ 18.255	R\$ 1.943	R\$ 16.312	89,4%
Otimista — 127 clientes (~18 meses)	R\$ 42.315	R\$ 3.705	R\$ 38.610	91,2%

Margem bruta cobre apenas custos de infraestrutura. Custos de equipe, marketing e impostos reduzem a margem líquida — especialmente nos primeiros 12 meses.

9.5 Margem por Plano no Cenário Moderado (59 clientes)

Plano	Receita	Custo Variável	Custo Fixo / cliente	Lucro Bruto
Starter (100 alunos)	R\$ 199	R\$ 12	R\$ 11	R\$ 176 (88%)
Escola (250 alunos)	R\$ 449	R\$ 30	R\$ 11	R\$ 408 (91%)
Rede (700 alunos)	R\$ 890	R\$ 84	R\$ 11	R\$ 795 (89%)

9.6 Alertas e Oportunidades

Ponto de atenção	Detalhe
Push FCM é gratuito — custo variável mínimo	FCM é gratuito e ilimitado. SMS restrito a OTP de ativação (~1-2 por responsável). Custo variável total por escola é inferior a R\$2/mês no plano Starter. Risco financeiro de abuso é praticamente nulo.
Câmbio do dólar	Toda a infra AWS é cobrada em USD. Alta de 10% no dólar representa aumento de ~R\$60/mês nos custos fixos. Mitigação: precificação com reajuste anual pelo IPCA.
Free Tier AWS (primeiros 12 meses)	Conta AWS nova tem créditos e tier gratuito que podem reduzir o custo do MVP em 30-40% no primeiro ano — especialmente RDS e EC2.
Escala de Fargate	A partir de ~200 clientes simultâneos, adicionar uma 3ª task (+R\$92/mês) dobra a capacidade. Upgrade de RDS para db.t4g.medium (+R\$200/mês) só necessário acima de 500 clientes.
Margem bruta vs. líquida	As margens acima são brutas (apenas infra). A margem líquida real nos primeiros 18 meses será negativa ou próxima de zero após salários, marketing e impostos — isso é esperado e normal para um SaaS em crescimento.

10. Escopo do MVP

Para o lançamento inicial, o MVP deve conter:

✓ Incluído no MVP

- Módulo de importação inteligente (CSV/Excel com detecção automática de campos e tolerância a dados incompletos)
- Onboarding colaborativo: convite por SMS, ativação por OTP, correção de dados pelo responsável e convite de segundo responsável pelo app
- Perfis: Administrador, Professor, Secretaria e Responsável

- Criação e envio de comunicados (texto + imagem + PDF)
- Confirmação de leitura com log imutável
- Reenvio automático para não-lidos
- Módulo de formulários do responsável (justificativa de falta, atestado, saída antecipada)
- Formulários customizáveis pela escola com geração de protocolo automático
- Agenda escolar com notificações
- Feed do responsável com filtros
- Dashboard básico para a escola
- Relatório de leitura exportável em PDF
- App mobile (iOS + Android) + interface web

x Fora do MVP (roadmap)

- SMS como fallback (roadmap futuro — versão paga opcional por escola)
- Integração com Google / Apple Calendar
- API pública para integração com SGEs
- Módulo financeiro (mensalidades)
- Múltiplas unidades — suporte básico no MVP (unidade como sub-entidade da escola)
- SSO com Google Workspace / Microsoft 365

11. Guia de Desenvolvimento — Claude Code

Este documento está estruturado para ser entregue diretamente ao Claude Code como prompt de inicialização. Todas as decisões de produto, arquitetura, banco de dados, API e estrutura de projeto estão documentadas abaixo com nível de detalhe suficiente para iniciar o desenvolvimento sem ambiguidades.

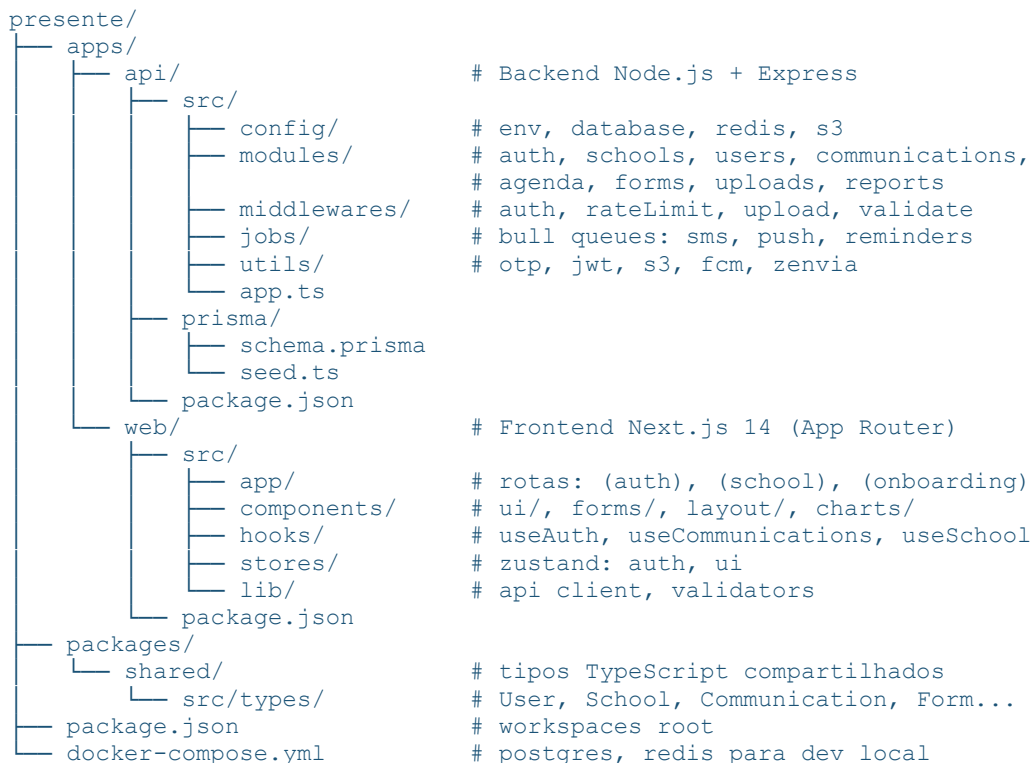
11.1 Instrução de Inicialização

Cole o seguinte prompt ao iniciar uma nova sessão no Claude Code:

```
Você vai desenvolver o Presente — plataforma SaaS B2B de comunicação
escola-família. Este documento contém todos os requisitos, arquitetura,
banco de dados e endpoints necessários. Siga exatamente o que está
especificado. Stack: Node.js + TypeScript (backend), React + Next.js
(web), PostgreSQL (banco), Redis (cache), AWS S3 (arquivos).
Inicie pelo backend: configure o monorepo, instale dependências,
crie o schema do banco e implemente autenticação JWT + OTP.
Use as seções 11 a 14 deste documento como fonte da verdade.
```

12. Estrutura do Projeto (Monorepo)

Monorepo gerenciado com npm workspaces. Todos os pacotes compartilham tipos TypeScript via pacote `@presente/shared`.



12.1 Dependências Principais

Pacote	Versão	Uso
express	^4.18	HTTP server + roteamento
prisma + @prisma/client	^5.x	ORM para PostgreSQL
jsonwebtoken	^9.x	JWT de autenticação (escola)
ioredis	^5.x	Cache + sessões + filas
bullmq	^4.x	Filas de SMS, push e lembretes
@aws-sdk/client-s3	^3.x	Upload de arquivos para S3
multer	^1.x	Middleware de upload
zod	^3.x	Validação de schemas
csv-parse	^5.x	Parser de planilhas importadas
next.js	^14.x	Frontend web (App Router)

react-query / tanstack	⁵ .x	Server state no frontend
tailwindcss	³ .x	Estilização frontend
zustand	⁴ .x	Estado global (auth, UI)
firebase-admin	¹¹ .x	Envio de push via FCM

13. Modelo de Entidades e Esquema do Banco de Dados

ORM: Prisma. Banco: PostgreSQL 15. Arquitetura multi-tenant por school_id em todas as tabelas. UUID v4 como chave primaria. Timestamps automaticos em todas as entidades.

13.1 Hierarquia de Entidades

A plataforma segue uma hierarquia clara de responsabilidades, acesso e comunicacao:

```

ESCOLA
|
|-- COORDENADOR (varios por escola)
|   |
|   |-- PROFESSOR (varios por coordenador)
|   |   |
|   |   |-- TURMA (um ou mais professores, 1 coordenador)
|   |   |   |
|   |   |   |-- ALUNO (varios por turma)
|   |   |   |   |
|   |   |   |   |-- RESPONSVEL (varios por aluno)
|   |   |   |   |
|   |   |   |   |-- COMUNICADO (entidade independente)
|   |   |   |   |   |
|   |   |   |   |   |-- Criado pela ESCOLA → alcance: TURMA ou ALUNO
|   |   |   |   |   |-- Criado pelo RESPONSVEL → alcance: ESCOLA (secretaria/prof)
|   |   |   |   |   |
|   |   |   |   |   |-- CONFIRMACAO DE VISUALIZACAO
|   |   |   |   |   |   |-- Simples: Li e estou ciente
|   |   |   |   |   |   |-- Imutavel apos confirmar
|   |   |   |   |   |   |-- Gera protocolo automatico
|   |   |   |   |   |   |
|   |   |   |   |   |   |-- ANEXOS (fotos, PDFs, documentos)

```

13.2 O Comunicado como Entidade Central

O Comunicado e uma entidade de primeira classe — independente dos destinatarios. Ele e criado com suas caracteristicas proprias, depois direcionado. Formularios do responsavel (justificativas, atestados, saidas antecipadas) sao comunicados de tipo especifico, nao uma entidade separada. Isso unifica o modelo e simplifica a UX.

Dimensao	Escola cria	Responsavel cria
Tipos disponiveis	NOTICE URGENT INFORMATIVE DOCUMENT PHOTO	ABSENCE MEDICAL_CERT EARLY_DEPARTURE
Alcance	TURMA (todos os responsaveis) ou ALUNO (especifico)	SCHOOL (secretaria + prof responsavel)
Confirmacao	Exigida — Li e estou ciente (imutavel)	Protocolo automatico gerado (ANO-SEQ4D)
Anexos	Fotos, PDFs, audios, videos	PDF ou foto do documento (atestado, etc)

Status	DRAFT SCHEDULED SENT CANCELLED	SENT RECEIVED UNDER_REVIEW RESOLVED
---------------	--------------------------------------	---

13.3 Enums

```
enum UserRole          { ADMIN | SECRETARY | COORDINATOR | TEACHER }
enum PlanType          { STARTER | SCHOOL | NETWORK | ENTERPRISE }

-- Comunicado: criado pela escola
enum SchoolCommType    { NOTICE | URGENT | INFORMATIVE | DOCUMENT | PHOTO | EXAM | MEETING | AUTHORIZATION | AUTHORIZATION }

-- Comunicado: criado pelo responsavel
enum GuardianCommType  { ABSENCE | MEDICAL_CERT | EARLY_DEPARTURE }

-- Alcance do comunicado
enum CommScope         { CLASS | STUDENT }

-- Status: comunicados da escola
enum SchoolCommStatus  { DRAFT | SCHEDULED | SENT | CANCELLED }

-- Status: comunicados do responsavel
enum GuardianCommStatus { SENT | RECEIVED | UNDER_REVIEW | RESOLVED }

enum LinkStatus        { PENDING_INVITE | PENDING_APPROVAL | ACTIVE | SUSPENDED }
```

13.4 Permissões por Role

Role	Permissões
ADMIN	Tudo — configuracoes, usuarios, importacao, todos os comunicados, relatorios
SECRETARY	Comunicados gerais, gestao de responsaveis, recebe comunicados dos responsaveis, relatorios
COORDINATOR	Comunicados das turmas sob sua coordenacao, supervisao dos professores, recebe comunicados dos responsaveis das suas turmas
TEACHER	Comunicados das turmas em que e professor titular (is_homeroom), recebe comunicados dos responsaveis dos seus alunos
GUARDIAN	Le feed de comunicados da escola, confirma leitura, cria comunicados do tipo responsavel (justificativas, atestados, saida antecipada)

13.5 Tabelas Detalhadas

1. schools — Escola

Entidade raiz (tenant). Todos os dados pertencem a uma escola. Isolamento total via school_id.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
name	VARCHAR(200)	Nome oficial — NOT NULL
cnpj	VARCHAR(18)	UNIQUE — opcional
logo_url	TEXT	URL no S3
address	VARCHAR(300)	Endereco completo
city	VARCHAR(100)	Cidade
state	CHAR(2)	UF
phone	VARCHAR(20)	Telefone institucional
email	VARCHAR(200)	E-mail institucional
plan	PlanType ENUM	DEFAULT STARTER
trial_ends_at	TIMESTAMP	NULL = fora do trial
active	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- 1 Escola → N Units (unidades) | 1 Escola → N Users
- 1 Escola → N Classes
- 1 Escola → N Students
- 1 Escola → N Communications

1a. school_units — Unidades da Escola

Sub-entidade da escola. Uma escola pode ter N unidades (ex: Unidade Centro, Unidade Zona Sul). Cada unidade tem suas proprias turmas e funcionarios mas compartilha o mesmo contrato e painel admin.

Campo	Tipo	Descricao
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id ON DELETE CASCADE — NOT NULL
name	VARCHAR(200)	Nome da unidade — NOT NULL — ex: Unidade Centro

address	VARCHAR (300)	Endereco especifico da unidade
city	VARCHAR (100)	Cidade
state	CHAR (2)	UF
phone	VARCHAR (20)	Telefone da unidade
active	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- N Units → 1 Escola
- 1 Unit → N Classes (turmas desta unidade)
- 1 Unit → N Users (staff alocado nesta unidade)

Nota: Admin da escola gerencia todas as unidades. Coordenador e professor podem ser alocados em uma ou mais unidades.

Campos adicionados em outras tabelas para suporte a multiplas unidades:

Tabela.campo	Tipo	Descricao
classes.unit_id	UUID FK	→ school_units.id — NULL = escola sem unidades
users.unit_id	UUID FK	→ school_units.id — NULL = usuario atua em todas as unidades

Cadastro hierarquico de staff (RF-01): Admin cadastra Diretor → Diretor cadastra Coordenadores → Coordenador cadastra Professores. Cada nivel so enxerga e gerencia o nivel abaixo. Diretor tem role=ADMIN com permissao restrita a sua unidade.

Visibilidade e Envio em Lote por Perfil

Regras de visibilidade de comunicados e envio em lote por perfil de usuario:

Perfil	Pode enviar comunicado para	Visualiza comunicados de
Admin/Diretora	Todas as turmas de todas as unidades — em lote	Todos os comunicados da escola
Secretaria	Todas as turmas da escola — em lote	Todos os comunicados da escola

Coordenadora	Todas as turmas sob sua coordenacao — em lote	Comunicados de suas turmas + comunicados dos responsaveis dessas turmas
Professor	Somente as turmas em que e professor (is_homeroom)	Comunicados de suas turmas + comunicados dos responsaveis dos seus alunos
Responsavel	Somente referente ao seu filho (atestado, falta, saida antecipada)	Comunicados da escola referentes ao seu filho

2. users — Staff da Escola

Tabela unificada para Admin, Secretary, Coordinator e Teacher. Role define permissões e acesso.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id ON DELETE CASCADE — NOT NULL
name	VARCHAR(200)	Nome completo — NOT NULL
email	VARCHAR(200)	UNIQUE por escola — login
password_hash	TEXT	bcrypt — NOT NULL
role	UserRole ENUM	ADMIN SECRETARY COORDINATOR TEACHER
phone	VARCHAR(20)	Telefone interno — opcional
avatar_url	TEXT	Foto de perfil — opcional
active	BOOLEAN	DEFAULT true
last_login_at	TIMESTAMP	
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- N Users → 1 Escola
- 1 Coordinator → N Teachers (via coordinator_teachers)
- 1 Teacher → N Classes (via class_teachers)

3. coordinator_teachers — Vinculo Coordenador-Professor

Define quais professores estao sob supervisao de qual coordenador. Professor tem no maximo 1 coordenador por escola.

Campo	Tipo	Descricao / Constraint
coordinator_id	UUID FK	→ users.id (role=COORDINATOR) — NOT NULL

teacher_id	UUID FK	→ users.id (role=TEACHER) — NOT NULL
school_id	UUID FK	→ schools.id — integridade multi-tenant
assigned_at	TIMESTAMP	DEFAULT NOW()
PRIMARY KEY	(coordinator_id, teacher_id)	
UNIQUE	(teacher_id, school_id)	Professor tem 1 coordenador por escola

4. classes — Turmas

Agrupamento de alunos. Pertence a uma unidade da escola. Tem 1 coordenador responsavel e pode ter multiplos professores. Professor titular (is_homeroom) pode criar comunicados diretamente.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id — NOT NULL
coordinator_id	UUID FK	→ users.id (role=COORDINATOR) — responsavel
name	VARCHAR(100)	Ex: 3 ano A — NOT NULL
grade	VARCHAR(50)	Ex: 3 ano — nivel de ensino
shift	VARCHAR(20)	manha tarde integral noturno
year	INTEGER	Ano letivo — ex: 2026
room	VARCHAR(20)	Sala — opcional
active	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- N Turmas → 1 Escola
- N Turmas → 1 Coordenador
- N:N Turmas com Professores (via class_teachers)
- 1 Turma → N Alunos

5. class_teachers — Vinculo Turma-Professor

Professor pode lecionar em multiplas turmas. Turma pode ter multiplos professores por disciplina. Titular tem permissao de criar comunicados.

Campo	Tipo	Descricao / Constraint
class_id	UUID FK	→ classes.id — NOT NULL

teacher_id	UUID FK	→ users.id (role=TEACHER) — NOT NULL
school_id	UUID FK	→ schools.id — integridade multi-tenant
subject	VARCHAR(100)	Disciplina — ex: Matematica — opcional
is_homerom	BOOLEAN	DEFAULT false — true = professor titular, pode criar comunicados
assigned_at	TIMESTAMP	DEFAULT NOW()
PRIMARY KEY	(class_id, teacher_id)	

6. students — Alunos

Cadastro do aluno. Pertence a uma turma ativa. Soft delete preserva historico.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id — NOT NULL
class_id	UUID FK	→ classes.id — turma principal — NOT NULL
name	VARCHAR(200)	Nome completo — NOT NULL
enrollment_code	VARCHAR(50)	Matricula do SGE — UNIQUE por escola
birth_date	DATE	Data de nascimento — opcional
gender	VARCHAR(20)	masculino feminino outro nao_informado — opcional
notes	TEXT	Observacoes internas: necessidades especiais, alergias, etc
photo_url	TEXT	Foto no S3 — opcional
active	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- N Students → 1 Class
- N Students → 1 Escola
- 1 Student → N Guardians (via student_guardians)

6a. student_classes — Vinculo Aluno-Turma (Pivot)

Um aluno pode estar em mais de uma turma simultaneamente. A class_id na tabela students representa a turma principal para comunicacao padrao.

Campo	Tipo	Descricao
student_id	UUID FK	→ students.id — NOT NULL
class_id	UUID FK	→ classes.id — NOT NULL
school_id	UUID FK	→ schools.id — integridade multi-tenant
is_primary	BOOLEAN	DEFAULT false — true = turma principal (coincide com students.class_id)
enrolled_at	TIMESTAMP	DEFAULT NOW()
active	BOOLEAN	DEFAULT true — false = aluno saiu desta turma

PRIMARY KEY	(student_id, class_id)	
-------------	---------------------------	--

scope=CLASS busca alunos via student_classes — garante que alunos em multiplas turmas recebam comunicados de todas elas.

7. guardians — Responsaveis

Pessoa autorizada a receber comunicados e criar comunicados do tipo responsavel. Autenticacao exclusivamente por OTP. Acesso via app mobile/web proprio.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id — NOT NULL
name	VARCHAR(200)	Preenchido na ativacao — NOT NULL
phone	VARCHAR(20)	UNIQUE global — OTP e identificacao principal
email	VARCHAR(200)	Preenchido na ativacao — opcional
push_token	TEXT	FCM token — atualizado a cada login — NULL = sem app
device_type	VARCHAR(10)	ios android web
avatar_url	TEXT	Foto de perfil — opcional
active	BOOLEAN	DEFAULT true
activated_at	TIMESTAMP	NULL ate completar o primeiro acesso
last_seen_at	TIMESTAMP	Ultima vez que abriu o app
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	AUTO

Relacionamentos:

- N Guardians → 1 Escola
- N:N Guardians com Students (via student_guardians)

Nota: Phone e UNIQUE global. Responsavel pode estar vinculado a mais de um aluno da mesma escola.

8. student_guardians — Vinculo Aluno-Responsavel

Pivot com relacionamento e status. Responsavel principal recebe todos os comunicados. Vinculo aprovado automaticamente se dados batem com importacao.

Campo	Tipo	Descricao / Constraint
student_id	UUID FK	→ students.id — NOT NULL

guardian_id	UUID FK	→ guardians.id — NOT NULL
school_id	UUID FK	→ schools.id — integridade multi-tenant
relationship	VARCHAR(50)	mae pai avo avo_paterno padrasto madrastra responsavel outro
is_primary	BOOLEAN	DEFAULT false — responsavel principal recebe todos os comunicados
status	LinkStatus ENUM	PENDING_INVITE PENDING_APPROVAL ACTIVE SUSPENDED
invited_by	UUID FK	→ guardians.id — NULL se escola importou
invited_at	TIMESTAMP	
activated_at	TIMESTAMP	NULL ate aceitar o convite
approved_by	UUID FK	→ users.id — aprovacao manual quando ha divergencia
approved_at	TIMESTAMP	
PRIMARY KEY	(student_id, guardian_id)	
UNIQUE	(student_id, guardian_id)	

Relacionamentos:

- N:N Students e Guardians
- Maximo 5 responsaveis por aluno (validado na aplicacao)

9. communications — Comunicados (Entidade Central)

Entidade independente que centraliza TODA comunicacao da plataforma — da escola para responsaveis E do responsavel para a escola. O tipo (school_type / guardian_type) e o criador (created_by / guardian_id) determinam o fluxo.

Campo	Tipo	Descricao
id	UUID PK	gen_random_uuid()
school_id	UUID FK	→ schools.id — NOT NULL
		— IDENTIFICACAO DO AUTOR —
created_by	UUID FK	→ users.id — preenchido quando escola cria (Admin/Coord/Teacher)
guardian_id	UUID FK	→ guardians.id — preenchido quando responsavel cria
		— TIPO: apenas um dos dois e preenchido —
school_type	SchoolCommType	NOTICE URGENT INFORMATIVE DOCUMENT PHOTO — NULL se guardian
guardian_type	GuardianCommType	ABSENCE MEDICAL_CERT EARLY_DEPARTURE — NULL se escola
		— CONTEUDO —
title	VARCHAR(300)	Titulo — NOT NULL
body	TEXT	Conteudo em texto — NOT NULL
		— ALCANCE —
scope	CommScope ENUM	CLASS = turma(s) STUDENT = aluno(s) especifico(s)
		— STATUS —
school_status	SchoolCommStatus	DRAFT SCHEDULED SENT CANCELLED — NULL se guardian
guardian_status	GuardianCommStatus	SENT RECEIVED UNDER_REVIEW RESOLVED — NULL se escola
		— CONFIRMACAO DE LEITURA —
requires_confirmation	BOOLEAN	DEFAULT true para escola, false para responsavel (gera protocolo)
protocol_number	VARCHAR(30)	UNIQUE — gerado para comunicados do responsavel: ANO-SEQ4D
internal_note	TEXT	Nota interna da escola ao resolver — invisivel ao responsavel
resolved_at	TIMESTAMP	Quando escola resolve comunicado do responsavel
resolved_by	UUID FK	→ users.id
		— LEMBRETES (apenas comunicados da escola) —

auto_reminder	BOOLEAN	DEFAULT true — push para não lidos após 24h
reminder_count	INTEGER	DEFAULT 0 — lembretes enviados — max 2
		— AGENDAMENTO —
scheduled_at	TIMESTAMP	NULL = envio imediato ao publicar
sent_at	TIMESTAMP	Preenchido ao mudar status para SENT
cancelled_at	TIMESTAMP	
cancelled_by	UUID FK	→ users.id
created_at	TIMESTAMP	DEFAULT NOW()

Relacionamentos:

- N Communications → 1 Escola
- scope=CLASS: N:N com Classes (via communication_classes)
- scope=STUDENT: N:N com Students (via communication_students)
- 1 Communication → N Attachments (anexos)
- 1 Communication → N communication_reads (log imutável — apenas para comunicados da escola)

Regras críticas: (1) created_by e guardian_id são mutuamente exclusivos — apenas um preenchido. (2) school_type e guardian_type são mutuamente exclusivos. (3) protocol_number preenchido apenas para comunicados do responsável. (4) Comunicado SENT não pode ser editado. (5) communication_reads só existe para comunicados da escola.

9a. communication_classes — Destinatários por Turma

Turmas que recebem comunicado com scope=CLASS. Todos os responsáveis ativos das turmas recebem.

Campo	Tipo	Descrição / Constraint
communication_id	UUID FK	→ communications.id
class_id	UUID FK	→ classes.id
PRIMARY KEY	(communication_id, class_id)	

9b. communication_students — Destinatários Individuais

Alunos que recebem comunicado com scope=STUDENT. Todos os responsáveis do aluno recebem.

Campo	Tipo	Descrição / Constraint
communication_id	UUID FK	→ communications.id
student_id	UUID FK	→ students.id

PRIMARY KEY	(communication_id, student_id)	
-------------	-----------------------------------	--

9c. communication_reads — Confirmacao de Leitura (Imutavel)

Registro permanente de confirmacao (ciencia ou autorizacao). Sem UPDATE. Sem DELETE. Apenas INSERT. Evidencia juridica e administrativa. Existe apenas para comunicados criados pela escola.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
communication_id	UUID FK	→ communications.id — NOT NULL
guardian_id	UUID FK	→ guardians.id — NOT NULL
student_id	UUID FK	→ students.id — qual filho originou a leitura
read_at	TIMESTAMP	NOT NULL — momento exato
device_type	VARCHAR(10)	ios android web
ip_address	VARCHAR(45)	IPv4 ou IPv6 — para auditoria
UNIQUE	(communication_id, guardian_id)	Um registro por responsavel por comunicado

Nota: Endpoint retorna 409 se ja existir. Sem endpoint de UPDATE ou DELETE para esta tabela.

10. attachments — Anexos

Arquivos vinculados a comunicados: fotos, PDFs, documentos. Armazenados no S3, entregues via CloudFront. Validos tanto para comunicados da escola quanto do responsavel.

Campo	Tipo	Descricao / Constraint
id	UUID PK	gen_random_uuid()
communication_id	UUID FK	→ communications.id
type	VARCHAR(20)	image pdf audio video
s3_key	TEXT	Chave no bucket S3 — NOT NULL
url	TEXT	URL publica CloudFront CDN
size_bytes	INTEGER	Tamanho em bytes
mime_type	VARCHAR(100)	Ex: image/jpeg, application/pdf
created_at	TIMESTAMP	DEFAULT NOW()

13.6 Índices Essenciais

```
-- Multi-tenant
CREATE INDEX idx_users_school ON users(school_id, role);
CREATE INDEX idx_classes_school ON classes(school_id, active);
CREATE INDEX idx_classes_coordinator ON classes(coordinator_id);
CREATE INDEX idx_students_school ON students(school_id, active);
CREATE INDEX idx_students_class ON students(class_id);
CREATE INDEX idx_guardians_phone ON guardians(phone);

-- Vinculos
CREATE INDEX idx_ct_teacher ON coordinator_teachers(teacher_id);
CREATE INDEX idx_clst_teacher ON class_teachers(teacher_id);
CREATE INDEX idx_sg_guardian ON student_guardians(guardian_id, status);
CREATE INDEX idx_sg_student ON student_guardians(student_id, status);

-- Comunicados
CREATE INDEX idx_comm_school ON communications(school_id, school_status);
CREATE INDEX idx_comm_guardian_src ON communications(guardian_id);
CREATE INDEX idx_comm_protocol ON communications(protocol_number);
CREATE INDEX idx_reads_comm ON communication_reads(communication_id);
CREATE INDEX idx_reads_guardian ON communication_reads(guardian_id);
CREATE INDEX idx_comm_sent_at ON communications(school_id, sent_at DESC);
```

13.7 Regras de Integridade da Aplicacao

Regra	Implementacao
Autor mutuamente exclusivo	created_by e guardian_id sao mutuamente exclusivos. Backend valida: se created_by preenchido, guardian_id deve ser NULL e vice-versa.
Tipo mutuamente exclusivo	school_type e guardian_type sao mutuamente exclusivos. Backend valida consistencia com o autor.
Protocolo automatico	protocol_number gerado automaticamente (ANO-SEQ4D) apenas para comunicados com guardian_id. Imutavel apos geracao.
Confirmacao apenas da escola	communication_reads existe apenas para comunicados com created_by preenchido. Backend rejeita tentativa de confirmar comunicado do responsavel.
Log imutavel	communication_reads: apenas INSERT. Retorna 409 se ja existir. Sem UPDATE, sem DELETE.
SENT e imutavel	Comunicado da escola com school_status=SENT: apenas cancelar (cancels_at). Comunicado do responsavel com guardian_status=SENT: apenas escola pode mudar status.
Hierarquia de comunicados	Professor so cria comunicado para turmas em que is_homeroom=true. Coordinator para turmas sob sua coordenacao. Admin e Secretary para qualquer turma.
Soft delete universal	Nunca DELETE fisico. Sempre active=false ou status de cancelamento.

14. API REST — Endpoints Completos

Base URL: /api/v1. Autenticação via Bearer token JWT no header Authorization. Todas as respostas em JSON. Erros seguem o padrão: { error: string, code: string, details?: any }.

14.1 Autenticação

Método	Endpoint	Auth	Descrição
POST	/auth/login	Público	Login staff escola — email + senha → JWT
POST	/auth/otp/send	Público	Envia OTP por SMS para o responsável
POST	/auth/otp/verify	Público	Valida OTP → retorna JWT do responsável
POST	/auth/refresh	JWT	Refresh do token JWT
POST	/auth/logout	JWT	Invalida token no Redis

14.2 Escola e Onboarding

Método	Endpoint	Auth	Descrição
POST	/schools/register	Público	Cria escola + admin → inicia trial 30 dias
GET	/schools/me	JWT Admin	Dados da escola atual
PUT	/schools/me	JWT Admin	Atualiza dados da escola
GET	/schools/me/stats	JWT Admin/Sec	Dashboard: alunos, responsáveis ativos, taxa leitura
POST	/schools/me/import	JWT Admin/Sec	Upload planilha → cria import_job → processa async
GET	/schools/me/import/:jobId	JWT Admin/Sec	Status e resultado do job de importação

14.3 Usuários (Staff da Escola)

Método	Endpoint	Auth	Descrição
GET	/users	JWT Admin	Lista usuários da escola
POST	/users	JWT Admin	Cria professor ou secretária
PUT	/users/:id	JWT Admin	Atualiza dados do usuário
DELETE	/users/:id	JWT Admin	Desativa usuário (soft delete)

14.4 Turmas e Alunos

Método	Endpoint	Auth	Descrição
--------	----------	------	-----------

GET	/classes	JWT Staff	Lista turmas (professor vê só as suas)
POST	/classes	JWT Admin	Cria turma
GET	/classes/:id/students	JWT Staff	Lista alunos da turma com status responsáveis
GET	/students	JWT Admin/Sec	Lista todos os alunos da escola
POST	/students	JWT Admin/Sec	Cria aluno individualmente
GET	/students/:id/guardians	JWT Staff	Lista responsáveis do aluno com status

14.5 Responsáveis

Método	Endpoint	Auth	Descrição
POST	/guardians/activate	JWT Guardian	Completa cadastro após OTP: nome, email, parentesco
GET	/guardians/me	JWT Guardian	Perfil do responsável logado
PUT	/guardians/me	JWT Guardian	Atualiza dados (email, push_token)
POST	/guardians/invite	JWT Guardian	Convida outro responsável via telefone
GET	/guardians/pending	JWT Admin/Sec	Lista responsáveis não ativados
POST	/guardians/:id/resend-invite	JWT Admin/Sec	Reenviar SMS de convite
POST	/guardians/link/:token/aprove	JWT Admin/Sec	Aprovação manual de vínculo com divergência

14.6 Comunicados

Método	Endpoint	Auth	Descrição
GET	/communications	JWT Staff	Lista comunicados (professor: só suas turmas)
POST	/communications	JWT Staff	Cria comunicado (draft ou envia imediatamente)
GET	/communications/:id	JWT Staff	Detalhes do comunicado
POST	/communications/:id/send	JWT Staff	Envia draft imediatamente
POST	/communications/:id/cancel	JWT Staff	Cancela comunicado enviado
GET	/communications/:id/reads	JWT Staff	Relatório de leitura por responsável
POST	/communications/:id/remind	JWT Staff	Reenviar lembrete manual para não lidos

GET	/communications/:id/report.pdf	JWT Staff	Exporta relatório de leitura em PDF
GET	/guardian/feed	JWT Guardian	Feed do responsável (filhos + não lidos primeiro)
POST	/guardian/communications/:id/read	JWT Guardian	Confirma leitura — cria communication_read

14.7 Agenda

Método	Endpoint	Auth	Descrição
GET	/events?from=&to=	JWT Staff	Lista eventos por período
POST	/events	JWT Staff	Cria evento + notifica responsáveis das turmas
PUT	/events/:id	JWT Staff	Atualiza evento
POST	/events/:id/cancel	JWT Staff	Cancela evento + notifica responsáveis
GET	/guardian/events	JWT Guardian	Agenda do responsável (filtrada pelos filhos)

14.8 Formulários

Método	Endpoint	Auth	Descrição
GET	/form-templates	JWT Guardian	Templates ativos disponíveis para o responsável
POST	/form-templates	JWT Admin	Cria template customizado
POST	/form-submissions	JWT Guardian	Responsável envia formulário → gera protocolo
GET	/form-submissions	JWT Staff	Lista submissões pendentes/resolvidas
PUT	/form-submissions/:id/resolve	JWT Staff	Resolve submissão + nota interna
GET	/guardian/form-submissions	JWT Guardian	Histórico de formulários do responsável

14.9 Upload de Arquivos

Método	Endpoint	Auth	Descrição
POST	/uploads/presigned-url	JWT Staff	Gera URL pré-assinada S3 para upload direto do browser

POST	/uploads/confirm	JWT Staff	Confirma upload concluído → cria attachment
POST	/uploads/form-attachment	JWT Guardian	Upload de atestado médico (multipart/form-data)

15. Fluxos de Autenticação

15.1 JWT — Staff da Escola

1. POST /auth/login { email, password }
2. Backend valida bcrypt → gera accessToken (1h) + refreshToken (7d)
3. refreshToken armazenado no Redis: key=refresh:{userId}, TTL=7d
4. Todas as rotas protegidas: Authorization: Bearer {accessToken}
5. Middleware extrai school_id do JWT payload → multi-tenant automático
6. POST /auth/refresh { refreshToken } → novo accessToken
Redis valida refreshToken → gera novo access → rotaciona refresh
7. POST /auth/logout → deleta refresh do Redis

JWT Payload: { sub: userId, school_id, role, iat, exp }

15.2 OTP — Responsável

1. POST /auth/otp/send { phone }
 - Gera código 6 dígitos → salva em otp_codes (TTL 10 min)
 - Envia SMS via Zenvia: 'Seu código Presente: 123456'
 - Rate limit: 3 tentativas por telefone por hora (Redis)
2. POST /auth/otp/verify { phone, code }
 - Busca otp_codes WHERE phone=? AND used=false AND expires_at>NOW()
 - Marca used=true
 - Busca guardian WHERE phone=? (cria se não existir)
 - Gera JWT: { sub: guardianId, school_id, role: GUARDIAN, iat, exp }
 - Retorna: { token, guardian, students[], isFirstAccess }
3. isFirstAccess=true → frontend redireciona para tela de ativação

16. Variáveis de Ambiente

Arquivo `.env.example` na raiz do projeto. Nunca commitar `.env` com valores reais.

```
# — App
NODE_ENV=development
PORT=3001
FRONTEND_URL=http://localhost:3000
API_URL=http://localhost:3001

# — Database
DATABASE_URL=postgresql://presente:senha@localhost:5432/presente_dev
DATABASE_URL_TEST=postgresql://presente:senha@localhost:5432/presente_test

# — Redis
REDIS_URL=redis://localhost:6379

# — JWT
JWT_SECRET=sua-chave-secreta-muito-longa-aqui
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=7d

# — AWS S3
AWS_REGION=sa-east-1
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
AWS_S3_BUCKET=presente-files-prod
AWS_CLOUDFRONT_DOMAIN=files.presente.com.br

# — Firebase FCM
FIREBASE_PROJECT_ID=presente-app
FIREBASE_PRIVATE_KEY=-----BEGIN PRIVATE KEY-----...
FIREBASE_CLIENT_EMAIL=firebase-adminsdk@presente-app.iam...

# — Zenvia SMS
ZENVIA_API_KEY=sua-api-key-zenvia
ZENVIA_SENDER=Presente# Zenvia usado apenas para OTP de ativacao. Push FCM para comunicados.

# — AWS SES
SES_REGION=sa-east-1
SES_FROM_EMAIL=noreply@presente.com.br
```

17. Setup de Desenvolvimento

17.1 Pré-requisitos

- Node.js 20+
- Docker + Docker Compose
- npm 10+
- Conta AWS (para S3 — local use MinIO)
- Conta Zenvia (para SMS — local use mock)

17.2 Comandos Iniciais

```
# 1. Clonar e instalar
git clone https://github.com/seu-usuario/presente.git
cd presente && npm install

# 2. Subir PostgreSQL + Redis localmente
docker-compose up -d

# 3. Configurar variáveis de ambiente
cp .env.example apps/api/.env
# editar apps/api/.env com suas credenciais

# 4. Rodar migrations e seed
cd apps/api
npx prisma migrate dev
npx prisma db seed

# 5. Iniciar em modo desenvolvimento
npm run dev # roda api + web em paralelo
# api: http://localhost:3001
# web: http://localhost:3000
```

17.3 Seed de Desenvolvimento

O seed cria dados realistas para desenvolvimento:

- 1 escola: Escola Girassol (plano Starter, trial ativo)
- 1 admin: admin@girassol.com.br / senha: Presente@2026
- 2 professores + 1 secretária
- 3 turmas: 1º ano A, 3º ano A, 4º ano B
- 15 alunos distribuídos nas turmas
- 20 responsáveis — 15 ativados, 5 pendentes
- 5 comunicados de exemplo (rascunho, enviado, com leitura parcial)
- 3 eventos de agenda
- 2 comunicados do responsavel pendentes de resolucao

17.4 Ordem de Implementação Recomendada

Siga esta ordem para que cada feature possa ser testada de forma independente:

Sprint	Módulo	Entregável
1	Infra + Auth	Monorepo, Docker, Prisma schema, migrations, JWT + OTP, seed
2	Escolas + Usuários	CRUD escola, registro, perfis de staff, CRUD turmas
3	Importação	Parser CSV/XLSX, detecção de campos, import_job, convite SMS
4	Responsáveis	Ativação OTP, vínculo aluno-responsável, convite entre responsáveis
5	Comunicados	CRUD, envio, push FCM, log de leitura, lembrete automático BulIMQ
6	Uploads S3	URLs pré-assinadas, confirmação, CloudFront CDN, attachments
7	Agenda	CRUD eventos, notificações antecipadas, feed do responsável
8	Formulários	Templates nativos + custom, submissão, protocolo, gestão
9	Dashboard + Reports	Stats, relatório PDF, adesão por turma, formulários pendentes
10	Frontend Web	Next.js — todas as telas do painel escolar (ver wireframes seção 11)

18. Regras para o Claude Code

Estas regras devem ser seguidas durante todo o desenvolvimento:

- **Multi-tenant obrigatório:** Toda query ao banco DEVE filtrar por school_id. Nunca retornar dados de outra escola. Middleware extractSchoolId valida isso em todas as rotas protegidas.
- **TypeScript strict:** tsconfig com strict: true. Sem any. Tipos compartilhados em @presente/shared.
- **Validação com Zod:** Todo request body validado com schema Zod antes de chegar no controller. Erros de validação retornam 400 com detalhes dos campos inválidos.
- **Log imutável de leitura:** communication_reads não tem UPDATE nem DELETE. Apenas INSERT. Endpoint de leitura retorna 409 se já existir registro para o mesmo par (communication_id, guardian_id).
- **Soft delete:** Nenhuma tabela usa DELETE real. Sempre active=false ou cancelled_at=NOW(). Queries filtram WHERE active=true por padrão.
- **Filas para I/O externo:** SMS, push e e-mail SEMPRE via BullMQ. Nunca chamada síncrona de provedor externo no request-response cycle. Retornar 202 Accepted.
- **Rate limiting:** OTP: 3 tentativas/telefone/hora. Upload: 20 arquivos/comunicado. API geral: 100 req/min por JWT via Redis.
- **Segurança de arquivos S3:** URLs pré-assinadas expiram em 60 min. Bucket privado — acesso apenas via CloudFront com signed cookies para responsáveis. Nunca expor URL direta do S3.
- **Protocolo de formulário:** Gerado automaticamente: ANO-SEQ4D. Ex: 2026-0042. Sequência por escola. Imutável após criação.
- **Testes:** Cada módulo com pelo menos testes unitários de services e testes de integração dos endpoints críticos (auth, communications, reads).

18.1 docker-compose.yml de Desenvolvimento

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: presente
      POSTGRES_PASSWORD: senha123
      POSTGRES_DB: presente_dev
    ports: ['5432:5432']
    volumes: ['pgdata:/var/lib/postgresql/data']
  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
volumes:
  pgdata:
```

19. Qualidade e Testes Automatizados

A cobertura de testes é requisito de produto, não opcional. O objetivo é garantir que nenhuma funcionalidade crítica quebre silenciosamente em produção, especialmente o log imutável de leitura, autenticação e envio de comunicados.

19.1 Stack de Testes

Camada	Ferramenta	Uso
Unitários (backend)	Vitest	Services, utils, validators, parsers de importação
Integração (backend)	Supertest + Vitest	Endpoints HTTP com banco de dados real (test DB)
E2E (web)	Playwright	Fluxos completos no browser — escola e responsável
Cobertura	c8 (via Vitest)	Meta: 80% de cobertura em módulos críticos
CI/CD	GitHub Actions	Roda testes a cada PR — bloqueia merge se falhar

19.2 Testes Unitários — Módulos Obrigatórios

Cada módulo do backend deve ter cobertura unitária dos seus services. Mocks para banco e provedores externos.

auth.service

- generateOtp() — formato 6 dígitos, unicidade, TTL correto
- verifyOtp() — código válido, expirado, já usado, telefone errado
- generateJwt() — payload correto (sub, school_id, role, exp)
- verifyJwt() — token válido, expirado, assinatura inválida
- refreshToken() — rotação correta, token antigo invalidado no Redis

communications.service

- create() — draft criado com campos corretos, school_id isolado
- send() — status muda para SENT, sent_at preenchido, job enfileirado
- cancel() — só cancela se status=SENT, cancelled_at preenchido
- confirmRead() — cria communication_read, retorna 409 se já existe (imutabilidade)
- sendReminder() — só envia para não-lidos, máximo 2x por comunicado
- getReadReport() — retorna lidos e pendentes corretamente por comunicado

import.service

- detectColumns() — reconhece variações de cabeçalho (Responsável 1, resp1, guardian_name)
- parseRow() — linha completa, linha sem telefone, linha sem turma
- importBatch() — alunos criados, duplicatas ignoradas por enrollment_code
- generateInvites() — convites apenas para responsáveis com telefone
- linkGuardian() — vínculo automático quando dados batem, manual quando diverge

forms.service

- submit() — protocolo gerado no formato ANO-SEQ4D, único por escola
- resolve() — status muda para RESOLVED, nota interna salva, invisível ao responsável
- getByGuardian() — retorna apenas formulários do responsável logado

guardians.service

- activate() — isFirstAccess=true na primeira ativação, campos obrigatórios validados
- inviteSecondGuardian() — vínculo criado com status PENDING_INVITE, SMS OTP disparado
- approveLink() — apenas admin/secretaria pode aprovar vínculos divergentes

19.3 Testes de Integração — Endpoints Críticos

Testam o endpoint HTTP completo com banco real (PostgreSQL de teste). Criados com Supertest + fixtures de seed.

Endpoint	Cenários testados
POST /auth/otp/send	Rate limit após 3 tentativas · número inválido retorna 400 · OTP salvo no banco
POST /auth/otp/verify	Código correto → JWT retornado · código errado → 401 · código expirado → 401 · isFirstAccess correto
POST /auth/login	Email/senha corretos → JWT · senha errada → 401 · usuário inativo → 403
POST /communications	Draft criado · school_id isolado (não cria em outra escola) · campos obrigatórios validados
POST /communications/:id/send	Status muda · job enfileirado · não pode reenviar cancelado
POST /guardian/communications/:id/read	Log criado · 409 se já lido · guardian de outra escola → 403
POST /form-submissions	Protocolo gerado · dados salvos · notificação enfileirada
POST /schools/me/import	Job criado · arquivo salvo no S3 · status processando retornado
GET /guardian/feed	Retorna só comunicados das turmas dos filhos do responsável · não-lidos primeiro

GET /communications/:id/reads	Apenas staff da mesma escola acessa · responsável de outra escola → 403
----------------------------------	---

19.4 Testes E2E — Playwright

Fluxos completos no browser Chrome/Firefox. Ambiente: Next.js + API rodando localmente com banco de teste.

Fluxos obrigatórios — Escola

- Onboarding completo: registro → criar turma → importar planilha → ver pré-visualização → confirmar
- Criar e enviar comunicado: preencher título + texto → selecionar turma → publicar → ver na lista com status
- Verificar relatório de leitura: abrir comunicado enviado → ver lista de lidos/pendentes → exportar PDF
- Gerenciar formulário recebido: ver lista pendente → abrir → resolver com nota interna
- Criar evento de agenda: preencher campos → selecionar turmas → salvar → ver no calendário
- Aprovar vínculo de responsável: ver pendentes → aprovar → responsável aparece como ativo

Fluxos obrigatórios — Responsável (app web simulado)

- Ativação: receber link → validar OTP → corrigir e-mail → confirmar parentesco → ver feed
- Ler comunicado: abrir feed → clicar em não lido → ler → tocar 'Li e estou ciente' → ver como lido
- Ver agenda: navegar para aba agenda → ver eventos do filho → checar datas corretas
- Enviar justificativa de falta: abrir formulários → selecionar falta → escolher data e motivo → enviar → ver protocolo
- Convidar segundo responsável: ir ao perfil → convidar → inserir telefone → confirmar

19.5 Regras de CI/CD

```
# .github/workflows/test.yml
on: [pull_request]
jobs:
  test-backend:
    - npm run test:unit          # vitest unitários
    - npm run test:integration  # supertest com DB de teste
    - npm run coverage          # falha se abaixo de 80% nos módulos críticos
  test-e2e:
    - npm run build              # build do Next.js
    - npx playwright test       # todos os fluxos E2E
  lint:
    - npm run lint              # ESLint + TypeScript check
# Merge bloqueado se qualquer job falhar
```

20. Suporte ao Usuário

A estratégia de suporte é reduzir ao máximo a necessidade de contato humano através de UX intuitiva, documentação inline e portal de ajuda completo. Meta: menos de 0,5 tickets de suporte por escola por mês.

20.1 Suporte Inline — Dentro do App

Cada funcionalidade do painel web e do app mobile tem suporte contextual embutido. O usuário nunca precisa sair do app para entender o que fazer.

Tooltips e ícones de ajuda

- Ícone '?' ao lado de cada campo complexo — exibe explicação em popover sem sair da tela
- Tooltip no botão 'Li e estou ciente' — explica o que o clique registra e sua validade legal
- Tooltip no log de leitura — explica o que é o protocolo e por que é imutável
- Tooltip na categoria 'Urgente' — explica que não pode ser silenciado pelo responsável
- Tooltip no campo de importação — lista formatos aceitos e link para download do template

Onboarding guiado (primeiro acesso)

- Escola: tour interativo de 5 passos ao criar a primeira turma — destaca os elementos da tela em sequência
- Escola: checklist de configuração visível no dashboard até completar: turmas criadas, alunos importados, primeiro comunicado enviado
- Responsável: tela de boas-vindas explica o que é o Presente e o que esperar do app antes de ver o feed
- Responsável: primeiro comunicado recebido tem destaque visual especial com instrução de como confirmar leitura

Empty states informativos

- Feed vazio: 'Nenhum aviso por enquanto. Você será notificado quando a escola enviar algo.'
- Lista de comunicados vazia: 'Nenhum comunicado enviado ainda. Clique em + Novo comunicado para começar.'
- Formulários sem pendências: 'Nenhuma solicitação pendente. Tudo em dia!'
- Agenda sem eventos: 'Nenhum evento próximo. Crie um evento para notificar os responsáveis.'

Mensagens de erro amigáveis

- Erros de validação em linguagem humana, não técnica — 'O título não pode ficar em branco' em vez de 'field required'
- Erros de rede com botão de retry e explicação simples
- OTP expirado: mensagem clara com botão para reenviar código
- Limite de upload atingido: informa o limite e sugere compactar a imagem

20.2 Portal Externo de Suporte

Portal público em ajuda.presente.com.br com artigos organizados por perfil de usuário e funcionalidade. Construído com Notion, Docusaurus ou similar.

Estrutura do portal

Seção	Artigos incluídos
Primeiros passos — Escola	Como criar minha conta · Importar alunos passo a passo · Criar turmas · Convidar professores · Enviar o primeiro comunicado
Comunicados	Como criar um comunicado · Tipos de categoria e quando usar cada um · Ver quem leu · Reenviar para quem não leu · Exportar relatório de leitura
Agenda	Como criar eventos · Tipos de evento · Como as notificações funcionam · Cancelar um evento
Formulários	Quais formulários estão disponíveis · Como o responsável envia · Como resolver uma solicitação · O que é o número de protocolo
Responsáveis	Como o responsável ativa o acesso · Como adicionar um segundo responsável · O que fazer se não recebeu o código · Como atualizar dados
Importação	Formatos aceitos · Como exportar do Escola Web / SAE / Totvs · Template oficial · O que fazer com alunos sem telefone
Segurança e privacidade	O que o Presente armazena · LGPD e dados de menores · Como revogar acesso de um responsável · Como exportar ou excluir dados
FAQ — Perguntas frequentes	Por que o responsável não recebeu a notificação · Posso editar um comunicado enviado · Como funciona o 'Li e estou ciente' legalmente · O app funciona sem internet

Vídeos curtos de onboarding

- Vídeo 1 (2 min): Como importar sua lista de alunos — screencast do fluxo completo
- Vídeo 2 (90s): Enviando seu primeiro comunicado
- Vídeo 3 (60s): Como o responsável ativa o acesso pelo celular
- Vídeo 4 (90s): Entendendo o relatório de leitura
- Vídeo 5 (60s): Como usar os formulários de justificativa

20.3 Canal de Suporte Humano (Escalonamento)

Suporte humano como último recurso, após esgotadas as opções de autoatendimento.

Nível	Canal	SLA de resposta
N1 — Dúvidas de uso	Chat no portal (bot com busca no portal)	Autoatendimento — imediato

N2 — Problema técnico	E-mail: suporte@presente.com.br	Resposta em ate 8h uteis
N3 — Urgente / Bug critico	WhatsApp dedicado (planos Escola+)	Resposta em ate 2h uteis
N4 — Onboarding assistido	Videochamada agendada	Incluso em planos Escola e Rede

21. Dashboards e Relatórios Operacionais

Visibilidade total sobre a saúde da comunicação da escola. O objetivo é que a diretora consiga avaliar em 30 segundos se a comunicação está funcionando bem — sem precisar abrir cada comunicado individualmente.

21.1 Dashboard Principal — Métricas Integradas

Expansão do dashboard existente (seção 3.6) com métricas de comunicação integradas ao painel principal.

Cards de KPI (linha superior)

Métrica	Cálculo	Alerta automático se
Taxa de adesão geral	Responsáveis ativos / total cadastrados	Menor que 80%
Taxa media de leitura (30d)	Media de leitura de todos os comunicados do mes	Menor que 70%
Comunicados enviados (mes)	COUNT comunicados com status=SENT no mes	Zero por mais de 7 dias
Formularios pendentes	COUNT form_submissions com status=PENDING	Maior que 10 pendentes
Responsaveis sem app	COUNT guardians com push_token IS NULL e activated	Maior que 15% da base

Gráficos integrados no dashboard principal

- Barra de adesão por turma — horizontal, ordenada da maior para menor, com threshold visual em 80%
- Taxa de leitura dos últimos 10 comunicados — linha do tempo, permite identificar queda de engajamento
- Formulários por status — donut chart: Pendente / Em análise / Resolvido
- Atividade semanal — heatmap de comunicados enviados por dia da semana nas últimas 8 semanas

21.2 Dashboard de Saúde da Comunicação

Tela dedicada acessível via menu 'Relatórios'. Visão aprofundada para diretoras e coordenadoras que querem acompanhar a comunicação com mais detalhe.

Painel de Engajamento

- Ranking de responsáveis mais engajados — os 10 que mais leram comunicados no mês

- Ranking de responsáveis menos engajados — os que têm mais comunicados não lidos pendentes
- Taxa de leitura por turma — comparativo entre todas as turmas da escola
- Tempo médio de leitura — quanto tempo após o envio os responsáveis costumam ler
- Taxa de leitura por categoria de comunicado — Urgente vs Aviso vs Informativo

Painel de Formulários

- Volume por tipo de formulário no mês — quais os mais usados (falta, atestado, saída antecipada)
- Tempo médio de resolução por tipo — benchmark para a equipe da escola
- Formulários resolvidos vs pendentes por semana — linha do tempo
- Responsáveis que mais usam formulários — permite identificar casos recorrentes

Painel de Adesão

- Evolução da adesão ao longo do tempo — gráfico de linha desde o primeiro dia
- Responsáveis pendentes de ativação — lista com nome, aluno, turma e dias desde o convite
- Botão de reenvio em lote — reenviar OTP para todos os pendentes com 1 clique
- Responsáveis sem push_token — lista de quem ativou mas nunca instalou o app corretamente

21.3 Relatórios Exportáveis

Relatorio	Formato	Conteudo
Leitura por comunicado	PDF + Excel	Lista completa: responsavel, aluno, turma, lido em, dispositivo.
Adesao da escola	PDF + Excel	Todos os responsaveis: status, data ativacao, turma, aluno, push_token ativo
Historico de comunicados	Excel	Todos os comunicados do periodo: titulo, categoria, turmas, enviado por, taxa de leitura
Formularios do periodo	Excel	Todas as submissoes: protocolo, tipo, responsavel, aluno, status, data resolucão
Engajamento mensal	PDF	Resumo executivo: KPIs do mes, comparativo com mes anterior, turmas destaque

21.4 Alertas Proativos

O sistema envia alertas automaticos para o admin/diretora — sem precisar entrar no dashboard para descobrir problemas.

- Push + e-mail: comunicado Urgente com taxa de leitura abaixo de 50% apos 4 horas
- E-mail semanal: resumo de adesao — quantos responsaveis ativaram na semana, quantos ainda pendentes
- Push: formulario com mais de 72h sem resolucão

- E-mail mensal: relatório de engajamento automático no primeiro dia do mês
- Push: responsável que ficou 30 dias sem abrir o app (risco de churn de engajamento)

21.5 Especificação Técnica dos Dashboards

Implementação no frontend Next.js com biblioteca de gráficos Recharts (já listada nas dependências).

```
# Endpoints de dados para os dashboards
GET /schools/me/stats          → KPIs gerais (dashboard principal)
GET /reports/engagement       → dados de engajamento (dashboard saúde)
GET /reports/forms-summary    → dados de formulários
GET /reports/adhesion         → dados de adesão + lista pendentes
GET /reports/communications   → histórico de comunicados do período
# Todos aceitam query params: ?from=YYYY-MM-DD&to=YYYY-MM-DD&class_id=

# Endpoints de exportação
GET /reports/communication/:id/export.pdf
GET /reports/adhesion/export.xlsx
GET /reports/communications/export.xlsx?from=&to=
GET /reports/forms/export.xlsx?from=&to=
GET /reports/monthly-summary.pdf?month=YYYY-MM
```

22. Infraestrutura como Código — Docker, Terraform e CI/CD

Esta seção especifica toda a infraestrutura do Presente como código. O objetivo é que o Claude Code possa provisionar, configurar e fazer deploy do produto sem decisões manuais ou ambiguidade.

22.1 Ambientes

O Presente opera em três ambientes com isolamento completo:

Ambiente	Branch	Descrição
development	qualquer branch local	Docker Compose local. Sem AWS. MinIO para S3, Redis e Postgres em container.
staging	develop	AWS real. Deploy automático a cada push na branch develop. URL: staging.presente.com.br
production	main	AWS real. Deploy manual aprovado via GitHub Actions. URL: app.presente.com.br

22.2 Docker

docker-compose.yml (desenvolvimento local completo)

Substitui o docker-compose basico da secao 17. Este e o arquivo completo de desenvolvimento:

```
version: '3.9'
services:

  postgres:
    image: postgres:15-alpine
    container_name: presente_postgres
    environment:
      POSTGRES_USER: presente
      POSTGRES_PASSWORD: presente_dev_pass
      POSTGRES_DB: presente_dev
    ports: ['5432:5432']
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./apps/api/prisma/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: [CMD-SHELL, 'pg_isready -U presente']
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: presente_redis
    ports: ['6379:6379']
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru
    volumes:
      - redisdata:/data
    healthcheck:
      test: [CMD, 'redis-cli', 'ping']
      interval: 5s
      retries: 5

  minio:
    image: minio/minio:latest
    container_name: presente_minio
    command: server /data --console-address ':9001'
    environment:
      MINIO_ROOT_USER: presente_minio
      MINIO_ROOT_PASSWORD: presente_minio_secret
    ports:
      - '9000:9000' # S3 API (usar como AWS_S3_ENDPOINT em dev)
      - '9001:9001' # Console web
    volumes:
      - miniodata:/data

  api:
    build:
      context: ./apps/api
      dockerfile: Dockerfile.dev
    container_name: presente_api
    ports: ['3001:3001']
    environment:
      NODE_ENV: development
```

```

    DATABASE_URL:
postgresql://presente:presente_dev_pass@postgres:5432/presente_dev
    REDIS_URL: redis://redis:6379
    AWS_S3_ENDPOINT: http://minio:9000
    AWS_S3_BUCKET: presente-local
    AWS_REGION: us-east-1
    AWS_ACCESS_KEY_ID: presente_minio
    AWS_SECRET_ACCESS_KEY: presente_minio_secret
volumes:
  - ./apps/api/src:/app/src
  - ./packages:/packages
depends_on:
  postgres: { condition: service_healthy }
  redis: { condition: service_healthy }
command: npm run dev

web:
  build:
    context: ./apps/web
    dockerfile: Dockerfile.dev
  container_name: presente_web
  ports: ['3000:3000']
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:3001
  volumes:
    - ./apps/web/src:/app/src
  command: npm run dev

volumes:
  pgdata:
  redisdata:
  miniodata:

```

apps/api/Dockerfile (producao)

Multi-stage build para imagem minima em producao:

```

# Stage 1: deps
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
COPY apps/api/package*.json ./apps/api/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci --workspace=apps/api --workspace=packages/shared

# Stage 2: builder
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build --workspace=apps/api
RUN npx prisma generate --schema=apps/api/prisma/schema.prisma

# Stage 3: runner (imagem final - apenas o necessario)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -S presente && adduser -S presente -G presente
COPY --from=builder /app/apps/api/dist ./dist
COPY --from=builder /app/apps/api/prisma ./prisma

```

```
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/packages/shared/dist ./packages/shared/dist
USER presente
EXPOSE 3001
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD wget -qO- http://localhost:3001/health || exit 1
CMD [node, 'dist/app.js']
```

apps/web/Dockerfile (producao)

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY apps/web/package*.json ./
RUN npm ci

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY apps/web .
ARG NEXT_PUBLIC_API_URL
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -S presente && adduser -S presente -G presente
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
USER presente
EXPOSE 3000
CMD [node, 'server.js']
```

22.3 Terraform — Infraestrutura AWS

Toda a infraestrutura AWS provisionada como código. Estrutura de diretórios:

```
infra/
  terraform/
    environments/
      staging/
        main.tf
        variables.tf
        terraform.tfvars
      production/
        main.tf
        variables.tf
        terraform.tfvars
    modules/
      networking/      # VPC, subnets, security groups
      database/        # RDS PostgreSQL
      cache/           # ElastiCache Redis
      ecs/             # Fargate cluster, services, task definitions
      storage/         # S3 bucket + CloudFront
      loadbalancer/    # ALB + target groups
      dns/             # Route53
      secrets/         # AWS Parameter Store
```

infra/terraform/modules/networking/main.tf

```
resource "aws_vpc" "presente" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "presente-${var.env}" }
}

# Subnets publicas (ALB)
resource "aws_subnet" "public" {
  count          = 2
  vpc_id         = aws_vpc.presente.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "presente-${var.env}-public-${count.index}" }
}

# Subnets privadas (ECS Fargate, RDS, ElastiCache)
resource "aws_subnet" "private" {
  count          = 2
  vpc_id         = aws_vpc.presente.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index + 10)
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = { Name = "presente-${var.env}-private-${count.index}" }
}

# Security Groups
resource "aws_security_group" "alb" {
  name = "presente-${var.env}-alb"
  vpc_id = aws_vpc.presente.id
  ingress { from_port=80 to_port=80 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
  ingress { from_port=443 to_port=443 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
  egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] }
```

```

}

resource "aws_security_group" "ecs" {
  name     = "presente-${var.env}-ecs"
  vpc_id   = aws_vpc.presente.id
  ingress {
    from_port     = 3001
    to_port       = 3001
    protocol       = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
  egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] }
}

resource "aws_security_group" "rds" {
  name     = "presente-${var.env}-rds"
  vpc_id   = aws_vpc.presente.id
  ingress {
    from_port     = 5432
    to_port       = 5432
    protocol       = "tcp"
    security_groups = [aws_security_group.ecs.id]
  }
}

```

infra/terraform/modules/database/main.tf

```

resource "aws_db_subnet_group" "presente" {
  name       = "presente-${var.env}"
  subnet_ids = var.private_subnet_ids
}

resource "aws_db_instance" "presente" {
  identifier      = "presente-${var.env}"
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = var.db_instance_class
  # staging: db.t4g.micro | production: db.t4g.small
  allocated_storage    = 20
  max_allocated_storage = 100
  db_name               = "presente"
  username              = "presente"
  password              = var.db_password
  db_subnet_group_name  = aws_db_subnet_group.presente.name
  vpc_security_group_ids = [var.rds_sg_id]
  multi_az              = var.env == "production"
  # Multi-AZ apenas em producao
  backup_retention_period = 7
  deletion_protection     = var.env == "production"
  skip_final_snapshot      = var.env != "production"
  tags = { Environment = var.env }
}

```

infra/terraform/modules/ecs/main.tf

```

resource "aws_ecs_cluster" "presente" {
  name = "presente-${var.env}"
  setting { name="containerInsights" value="enabled" }
}

```

```

resource "aws_ecs_task_definition" "api" {
  family           = "presente-api-${var.env}"
  network_mode     = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  cpu              = "512"      # 0.5 vCPU
  memory           = "1024"     # 1 GB
  execution_role_arn = aws_iam_role.ecs_execution.arn
  task_role_arn     = aws_iam_role.ecs_task.arn

  container_definitions = jsonencode([
    {
      name       = "api"
      image      = "${var.ecr_repo_api}:${var.image_tag}"
      essential = true
      portMappings = [{ containerPort=3001, protocol="tcp" }]
      environment = [
        { name="NODE_ENV",    value=var.env },
        { name="PORT",       value="3001" },
      ]
      secrets = [
        { name="DATABASE_URL",    valueFrom="/presente/${var.env}/DATABASE_URL" },
        { name="REDIS_URL",       valueFrom="/presente/${var.env}/REDIS_URL" },
        { name="JWT_SECRET",     valueFrom="/presente/${var.env}/JWT_SECRET" },
        { name="AWS_S3_BUCKET",   valueFrom="/presente/${var.env}/AWS_S3_BUCKET" },
        { name="FIREBASE_PROJECT_ID",
          valueFrom="/presente/${var.env}/FIREBASE_PROJECT_ID" },
        { name="ZENVIA_API_KEY",  valueFrom="/presente/${var.env}/ZENVIA_API_KEY" },
      ]
      logConfiguration = {
        logDriver = "awslogs"
        options = {
          awslogs-group      = "/ecs/presente-api-${var.env}"
          awslogs-region     = var.aws_region
          awslogs-stream-prefix = "api"
        }
      }
      healthCheck = {
        command = ["CMD-SHELL", "wget -qO- http://localhost:3001/health || exit
1"]
        interval = 30
        timeout = 5
        retries = 3
        startPeriod = 60
      }
    }
  ])
}

resource "aws_ecs_service" "api" {
  name           = "presente-api-${var.env}"
  cluster       = aws_ecs_cluster.presente.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count = var.env == "production" ? 2 : 1
  launch_type   = "FARGATE"

  network_configuration {
    subnets      = var.private_subnet_ids
    security_groups = [var.ecs_sg_id]
  }
}

```

```

load_balancer {
  target_group_arn = var.alb_target_group_arn
  container_name   = "api"
  container_port   = 3001
}

deployment_circuit_breaker {
  enable   = true
  rollback = true
  # Rollback automatico se o deploy falhar
}
}

# Auto Scaling
resource "aws_appautoscaling_target" "api" {
  max_capacity       = 4
  min_capacity       = var.env == "production" ? 2 : 1
  resource_id        =
"service/${aws_ecs_cluster.presente.name}/${aws_ecs_service.api.name}"
  scalable_dimension = "ecs:service:DesiredCount"
  service_namespace = "ecs"
}

resource "aws_appautoscaling_policy" "api_cpu" {
  name                = "presente-api-cpu-${var.env}"
  policy_type         = "TargetTrackingScaling"
  resource_id         = aws_appautoscaling_target.api.resource_id
  scalable_dimension = aws_appautoscaling_target.api.scalable_dimension
  service_namespace   = aws_appautoscaling_target.api.service_namespace
  target_tracking_scaling_policy_configuration {
    target_value = 70 # Scale quando CPU > 70%
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
  }
}
}

```

infra/terraform/modules/storage/main.tf

```

resource "aws_s3_bucket" "files" {
  bucket = "presente-files-${var.env}"
}

resource "aws_s3_bucket_public_access_block" "files" {
  bucket                = aws_s3_bucket.files.id
  block_public_acls     = true
  block_public_policy   = true
  ignore_public_acls    = true
  restrict_public_buckets = true
  # Bucket 100% privado - acesso via CloudFront apenas
}

resource "aws_cloudfront_origin_access_control" "files" {
  name                = "presente-files-${var.env}"
  origin_access_control_origin_type = "s3"
  signing_behavior    = "always"
  signing_protocol    = "sigv4"
}

resource "aws_cloudfront_distribution" "files" {

```

```

enabled          = true
price_class      = "PriceClass_All"
comment          = "Presente files CDN - ${var.env}"

origin {
  domain_name      = aws_s3_bucket.files.bucket_regional_domain_name
  origin_id        = "s3-presente-files"
  origin_access_control_id = aws_cloudfront_origin_access_control.files.id
}

default_cache_behavior {
  allowed_methods      = ["GET", "HEAD"]
  cached_methods      = ["GET", "HEAD"]
  target_origin_id     = "s3-presente-files"
  viewer_protocol_policy = "redirect-to-https"
  cache_policy_id      = "658327ea-f89d-4fab-a63d-7e88639e58f6"
  # Managed-CachingOptimized policy
}

restrictions {
  geo_restriction { restriction_type = "none" }
}

viewer_certificate {
  cloudfront_default_certificate = true
}
}

```

infra/terraform/environments/production/terraform.tfvars

```

env          = "production"
aws_region   = "sa-east-1"
vpc_cidr     = "10.0.0.0/16"
db_instance_class = "db.t4g.small"
domain       = "presente.com.br"
api_domain   = "app.presente.com.br"
files_domain = "files.presente.com.br"

# Imagens Docker (preenchidas pelo CI/CD)
ecr_repo_api   = "ACCOUNT.dkr.ecr.sa-east-1.amazonaws.com/presente-api"
ecr_repo_web   = "ACCOUNT.dkr.ecr.sa-east-1.amazonaws.com/presente-web"
image_tag      = "latest"

```

22.4 GitHub Actions — CI/CD

Tres workflows: CI (testes em todo PR), Deploy Staging (automatico na develop), Deploy Production (manual com aprovacao).

[.github/workflows/ci.yml](#) — Testes e Qualidade

```
name: CI

on:
  pull_request:
    branches: [main, develop]
  push:
    branches: [develop]

jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck

  test-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run test:unit --workspace=apps/api
      - run: npm run coverage --workspace=apps/api
      # Falha se cobertura abaixo de 80% nos modulos criticos

  test-integration:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: presente
          POSTGRES_PASSWORD: presente_test
          POSTGRES_DB: presente_test
        ports: ['5432:5432']
        options: --health-cmd pg_isready --health-interval 5s
      redis:
        image: redis:7-alpine
        ports: ['6379:6379']
        options: --health-cmd 'redis-cli ping' --health-interval 5s
    env:
      DATABASE_URL:
      postgresql://presente_test@localhost:5432/presente_test
      REDIS_URL: redis://localhost:6379
      NODE_ENV: test
      JWT_SECRET: test_secret_key_not_for_production
    steps:
      - uses: actions/checkout@v4
```

```

- uses: actions/setup-node@v4
  with: { node-version: '20', cache: 'npm' }
- run: npm ci
- run: npx prisma migrate deploy --schema=apps/api/prisma/schema.prisma
- run: npm run test:integration --workspace=apps/api

test-e2e:
  runs-on: ubuntu-latest
  needs: [test-unit, test-integration]
  services:
    postgres:
      image: postgres:15-alpine
      env:
        POSTGRES_USER: presente
        POSTGRES_PASSWORD: presente_test
        POSTGRES_DB: presente_test
      ports: ['5432:5432']
      options: --health-cmd pg_isready --health-interval 5s
    redis:
      image: redis:7-alpine
      ports: ['6379:6379']
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
    - run: npm ci
    - run: npx playwright install --with-deps chromium
    - name: Start API
      run: npm run start:test --workspace=apps/api &
      env:
        DATABASE_URL:
postgres://presente:presente_test@localhost:5432/presente_test
        REDIS_URL: redis://localhost:6379
        NODE_ENV: test
        JWT_SECRET: test_secret_key_not_for_production
    - name: Start Web
      run: npm run build && npm run start --workspace=apps/web &
      env:
        NEXT_PUBLIC_API_URL: http://localhost:3001
    - run: npx wait-on http://localhost:3001/health http://localhost:3000
    - run: npx playwright test
    - uses: actions/upload-artifact@v4
      if: failure()
      with:
        name: playwright-report
        path: playwright-report/

```

.github/workflows/deploy-staging.yml — Deploy Automatico

```

name: Deploy Staging

on:
  push:
    branches: [develop]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: staging
    needs: [] # CI ja passou no push para develop
    steps:

```

```

- uses: actions/checkout@v4

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: sa-east-1

- name: Login ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2

- name: Build e push API
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.sha }
  run: |
    docker build -t $ECR_REGISTRY/presente-api:$IMAGE_TAG \
      -f apps/api/Dockerfile .
    docker push $ECR_REGISTRY/presente-api:$IMAGE_TAG
    docker tag $ECR_REGISTRY/presente-api:$IMAGE_TAG \
      $ECR_REGISTRY/presente-api:staging-latest
    docker push $ECR_REGISTRY/presente-api:staging-latest

- name: Build e push Web
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.sha }
  run: |
    docker build -t $ECR_REGISTRY/presente-web:$IMAGE_TAG \
      --build-arg NEXT_PUBLIC_API_URL=https://api-staging.presente.com.br \
      -f apps/web/Dockerfile .
    docker push $ECR_REGISTRY/presente-web:$IMAGE_TAG

- name: Deploy no ECS
  run: |
    aws ecs update-service \
      --cluster presente-staging \
      --service presente-api-staging \
      --force-new-deployment
    aws ecs wait services-stable \
      --cluster presente-staging \
      --services presente-api-staging

- name: Rodar migrations em staging
  run: |
    aws ecs run-task \
      --cluster presente-staging \
      --task-definition presente-migrate-staging \
      --launch-type FARGATE \
      --network-configuration 'awsvpcConfiguration={
        subnets=["${ secrets.PRIVATE_SUBNET_1 }"],
        securityGroups=["${ secrets.ECS_SG }"]
      }'

- name: Notificar Slack
  if: always()
  uses: slackapi/slack-github-action@v1.26.0
  with:
    payload: |

```

```
      {"text": "Deploy staging: ${ job.status } | ${ github.sha } | ${ github.actor }"}
    env:
      SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

.github/workflows/deploy-production.yml — Deploy com Aprovacao

```
name: Deploy Production

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    # Environment 'production' configurado no GitHub com required reviewers
    # O deploy so avanca apos aprovacao manual de um reviewer cadastrado
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID_PROD }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY_PROD }
          aws-region: sa-east-1

      - name: Login ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build e push API (producao)
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          docker build -t $ECR_REGISTRY/presente-api:$IMAGE_TAG \
            -t $ECR_REGISTRY/presente-api:latest \
            -f apps/api/Dockerfile .
          docker push $ECR_REGISTRY/presente-api:$IMAGE_TAG
          docker push $ECR_REGISTRY/presente-api:latest

      - name: Build e push Web (producao)
        env:
          ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
          IMAGE_TAG: ${ github.sha }
        run: |
          docker build -t $ECR_REGISTRY/presente-web:$IMAGE_TAG \
            --build-arg NEXT_PUBLIC_API_URL=https://app.presente.com.br \
            -f apps/web/Dockerfile .
          docker push $ECR_REGISTRY/presente-web:$IMAGE_TAG

      - name: Rodar migrations em producao
        run: |
          aws ecs run-task \
            --cluster presente-production \
            --task-definition presente-migrate-production \
            --launch-type FARGATE \
```

```

        --network-configuration 'awsvpcConfiguration={
            subnets=["${ secrets.PRIVATE_SUBNET_1_PROD }"],
            securityGroups=["${ secrets.ECS_SG_PROD }"]
        }'

- name: Deploy API no ECS
  run: |
    aws ecs update-service \
      --cluster presente-production \
      --service presente-api-production \
      --task-definition presente-api-production:${ steps.task-def.outputs.task-definition }
    aws ecs wait services-stable \
      --cluster presente-production \
      --services presente-api-production
    # Circuit breaker: rollback automatico se servico nao estabilizar

- name: Criar tag de release no GitHub
  run: |
    gh release create v$(date +%Y%m%d-%H%M%S) \
      --title 'Release $(date +%Y-%m-%d)' \
      --notes 'Deploy automatico via GitHub Actions'
  env:
    GH_TOKEN: ${ secrets.GITHUB_TOKEN }

```

22.5 Secrets e Variaveis de Ambiente por Ambiente

Secrets armazenados no AWS Parameter Store (SecureString). GitHub Actions acessa via OIDC — sem armazenar AWS keys permanentes no GitHub.

Secret (Parameter Store path)	Ambiente	Como definir
/presente/staging/DATABASE_URL	staging	RDS endpoint gerado pelo Terraform
/presente/production/DATABASE_URL	production	RDS endpoint gerado pelo Terraform
/presente/{env}/REDIS_URL	ambos	ElastiCache endpoint gerado pelo Terraform
/presente/{env}/JWT_SECRET	ambos	Gerar com: openssl rand -base64 64
/presente/{env}/AWS_S3_BUCKET	ambos	presente-files-{env}
/presente/{env}/ZENVIA_API_KEY	ambos	Painel Zenvia — chave de produção
/presente/{env}/FIREBASE_PROJECT_ID	ambos	Console Firebase
/presente/{env}/FIREBASE_PRIVATE_KEY	ambos	JSON da service account Firebase
/presente/{env}/SES_FROM_EMAIL	ambos	noreply@presente.com.br (verificado no SES)

GitHub Secrets necessários (Settings > Secrets > Actions)

```

AWS_ACCESS_KEY_ID           # staging deploy
AWS_SECRET_ACCESS_KEY       # staging deploy
AWS_ACCESS_KEY_ID_PROD      # production deploy
AWS_SECRET_ACCESS_KEY_PROD  # production deploy
SLACK_WEBHOOK               # notificacoes de deploy
PRIVATE_SUBNET_1            # staging subnet ID
PRIVATE_SUBNET_1_PROD       # production subnet ID
ECS_SG                      # staging security group
ECS_SG_PROD                 # production security group

```

22.6 Estrategia de Branches (GitFlow simplificado)

Branch	Proposito	Regras
main	Código em produção	Protegida. Merge apenas via PR aprovado de develop. Dispara deploy produção.
develop	Integração contínua	Protegida. Merge via PR com CI passando. Dispara deploy staging automático.
feature/nome	Nova funcionalidade	Criada a partir de develop. PR para develop com aprovação de 1 revisor.

fix/nome	Correcao de bug	Criada a partir de develop (ou main para hotfix urgente).
hotfix/nome	Correcao urgente em prod	Criada a partir de main. PR simultaneo para main e develop.

22.7 Health Check e Monitoramento

Endpoint /health obrigatorio na API para o ALB e ECS verificarem a saude do container:

```
// apps/api/src/routes/health.ts
router.get('/health', async (req, res) => {
  const checks = {
    status: 'ok',
    timestamp: new Date().toISOString(),
    version: process.env.npm_package_version,
    database: 'unknown',
    redis: 'unknown',
  };
  try {
    await prisma.$queryRaw`SELECT 1`;
    checks.database = 'ok';
  } catch { checks.database = 'error'; }
  try {
    await redis.ping();
    checks.redis = 'ok';
  } catch { checks.redis = 'error'; }
  const healthy = checks.database === 'ok' && checks.redis === 'ok';
  res.status(healthy ? 200 : 503).json(checks);
});
```

CloudWatch Alarms configurados via Terraform para alertar quando: CPU do ECS acima de 80%, RDS connections acima de 80% do limite, taxa de erros 5xx acima de 1% nas ultimas 5 min, health check do ALB falhando.

22.8 Checklist de Pre-Launch (antes do primeiro deploy em producao)

- [] Repositorio ECR criado para api e web
- [] Terraform aplicado em producao — VPC, RDS, ElastiCache, ECS, ALB, S3, CloudFront provisionados
- [] Todos os secrets cadastrados no Parameter Store de producao
- [] Dominio presente.com.br apontando para o ALB via Route53
- [] Certificado SSL emitido pelo ACM para *.presente.com.br
- [] SES verificado para noreply@presente.com.br
- [] Firebase project criado e service account gerada
- [] Zenvia API key ativa e testada
- [] GitHub Environments configurados (staging e production) com required reviewers
- [] Branch main e develop protegidas — push direto bloqueado
- [] Primeiro deploy de staging executado e testado manualmente

- ☐ Prisma migrations rodadas em producao via ECS task
- ☐ Seed de producao executado (escola demo para onboarding)
- ☐ CloudWatch alarms configurados e testados
- ☐ Backup do RDS verificado — snapshot manual antes do launch